

Taller 2: Operaciones de transformación del histograma

IMÁGENES Y VISIÓN - ISIS4825

Profesor: PhD Juan Pablo Reyes

Autores: Estefanía Laverde, Harvy Benitez.

Nota: Se declara el uso de ChatGPT en la elaboración de la versión final de este Taller. Siguiendo la recomendación del profesor, se utilizó como base el código provisto en el enlace del Taller 2, dado que este facilita implementaciones específicas que resultan innecesariamente complejas cuando se generan desde cero con una herramienta de IA.

La IA generativa fue empleada principalmente para mejorar la redacción de los párrafos de algunas respuestas, pero haciendo el análisis por cuenta propia.

```
In [ ]: !pip3 install scikit-image --quiet
```

```
[notice] A new release of pip is available: 25.3 -> 26.0.1  
[notice] To update, run: pip3 install --upgrade pip
```

```
In [ ]: # librerías  
import cv2  
from skimage import exposure  
  
import numpy as np  
import matplotlib.pyplot as plt  
import os
```

1. Calibración del histograma (o expansión del contraste)

1.1. Cree una nueva sección en su notebook.

1.2. Cargue la imagen de trabajo y visualicela.

```
In [ ]: # funcion para cargar y visualizar imágenes
def read_and_show_image_gray_scale(image_path:str):
    """
    Lee una imagen en escala de grises y la muestra.

    Parámetros:
    image_path (str): Ruta de la imagen a leer.

    Retorna:
    image (numpy.ndarray): Imagen leída en escala de grises.
    """

    # leer la imagen
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    # mostrar la imagen
    plt.imshow(image, cmap='gray', vmin=0, vmax=255)
    plt.axis('off')
    plt.show()

    return image
```

```
In [ ]: # path a las imágenes
path = 'images/'

# miramos las imagenes del directorio
images = os.listdir(path)
print(images)
```

```
['MUSCLE.png', 'ANGIO.png', 'QUITO.png', 'RONDELLE.png', 'BOUGIES.png', 'BABOON.png']
```

```
In [ ]: quito_image = read_and_show_image_gray_scale(path + 'QUITO.png')
```



1.3. Visualice el histograma de la imagen. Describa su forma. ¿Cuáles son los niveles de gris mínimo y máximo?

```
In [ ]: # definimos una nueva función que calcula el histograma de una imagen
def calculate_histogram(image:np.ndarray, min_val:int = 0, max_val:int = 255, hist_title:str = "", show:bool = True) ->
    """
    Calcula el histograma de una imagen en escala de grises, lo normaliza y grafica el resultado.

    Parámetros:
    image (numpy.ndarray): Imagen en escala de grises.
    min_val (int): Valor mínimo para el rango del histograma.
    max_val (int): Valor máximo para el rango del histograma.
    hist_title (str): Título para el gráfico del histograma.

    Retorna:
    histogram (numpy.ndarray): Histograma de la imagen.
    """
    # calcular el histograma
    histogram = cv2.calcHist([image], [0], None, [256], [0, 256])
```

```
# normalizar el histograma
histogram = histogram / histogram.sum()

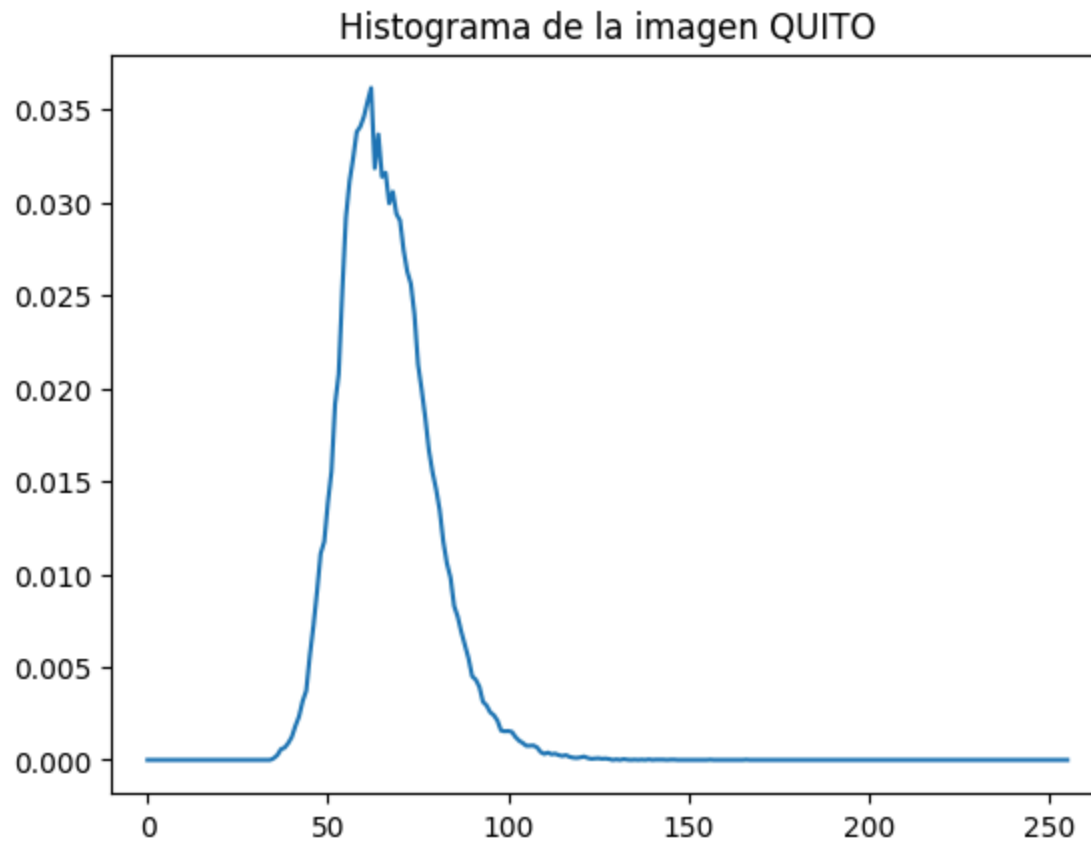
# plotear el histograma
plt.plot(histogram)
plt.xlim([min_val-10, max_val+10])
if hist_title:
    plt.title(hist_title)

# mostrar si show es True
if show:
    plt.show()

return histogram
```

```
In [ ]: # visualizamos el histograma de la imagen
histogram = calculate_histogram(quito_image, hist_title="Histograma de la imagen QUITO")

# pasar la imagen a un vector para calcular mínimo y máximo
print(f"Nivel de gris mínimo: {min(quito_image.flatten())}")
print(f"Nivel de gris máximo: {max(quito_image.flatten())}")
```



Nivel de gris mínimo: 35

Nivel de gris máximo: 166

El histograma muestra la distribución de los niveles de intensidad de los píxeles de la imagen, donde el eje x representa los valores de intensidad en escala de grises (de 0 a 255) y el eje y indica la cantidad de píxeles asociados a cada nivel de intensidad.

La imagen QUITO muestra que la gran mayoría de píxeles tienen valores de grises bajos, entre 40 y 90 aproximadamente. El valor mínimo es de 35, mientras que, el valor máximo es de 166. Esto se puede observar en la imagen, donde no hay un buen nivel de contraste y la imagen presenta niveles de gris medios-bajos. Este comportamiento refleja un uso limitado del rango dinámico, lo que se traduce en un contraste global reducido.

1.4. Efectúe una expansión del contrastete, visualice la imagen resultado y el histograma correspondiente. ¿En qué consiste la mejora de la imagen?

```
In [ ]: # expansion del contraste
rescaled_img_quito = exposure.rescale_intensity(quito_image, in_range=(35,166), out_range=(0,255)).astype(np.uint8)
```

```
# visualizamos la imagen original al lado de la imagen con expansión del contraste
plt.figure(figsize=(11,5))
plt.subplot(1,2,1)
plt.imshow(quito_image, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen QUITO original")

plt.subplot(1,2,2)
plt.imshow(rescaled_img_quito, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen QUITO con expansión del contraste")
plt.show()

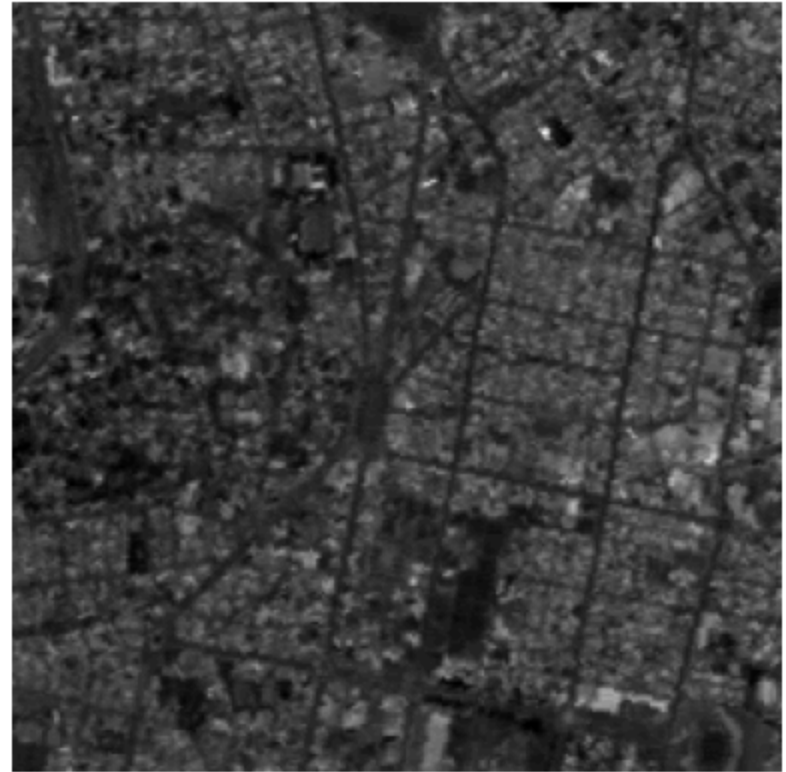
# visualizamos el histograma de la imagen resultado
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
histogram_original = calculate_histogram(quito_image, hist_title="Histograma QUITO original", show=False)
plt.subplot(1,2,2)
histogram_rescaled = calculate_histogram(rescaled_img_quito, hist_title="Histograma QUITO con expansión del contraste",
plt.show())

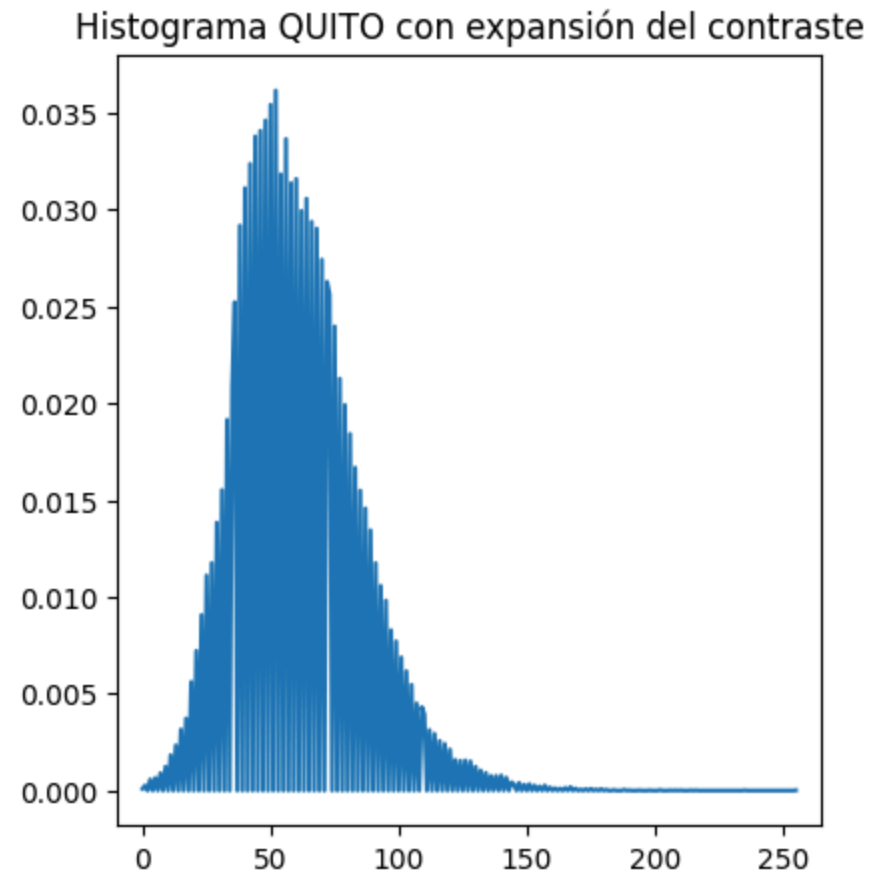
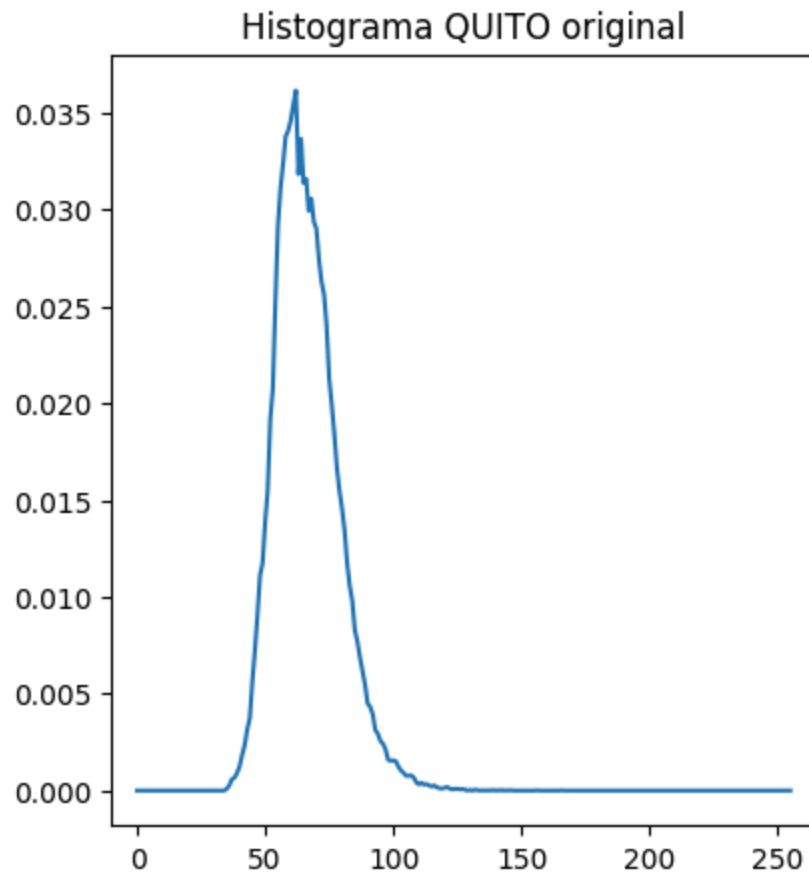
print("Valor minimo de grises en la imagen con expansión del contraste:", min(rescaled_img_quito.flatten()))
print("Valor maximo de grises en la imagen con expansión del contraste:", max(rescaled_img_quito.flatten()))
```

Imagen QUITO original



Imagen QUITO con expansión del contraste





Valor mínimo de grises en la imagen con expansión del contraste: 0
Valor máximo de grises en la imagen con expansión del contraste: 255

Al realizar una expansión del contraste, se toman los valores mínimos y máximos de la imagen y se reescalan de forma que el valor mínimo se convierta en 0 y el valor máximo se convierta en 255, mientras que, los valores inferiores y superiores del rango de origen se saturan hacia los extremos del nuevo rango

Esto mejora la imagen al aumentar el contraste (separación de niveles de grises en un intervalo más amplio), haciendo que los detalles sean más visibles y la imagen tenga mayor claridad. En el histograma, esto se reflejará en una expansión de la distribución de los valores de gris ocupando el rango completo de 0 a 255. Adicionalmente, aparecen saltos o huecos entre niveles consecutivos ya que el reescalado asigna los nuevos niveles de gris de forma discreta.

1.5. También es posible efectuar una expansión del contraste entre dos valores particulares. Se trata de una transformación lineal del histograma, que permite

especificar 2 niveles de gris que deben ser respectivamente llevados a 0 y 255. Esta transformación se llama calibración del histograma.

1.6. Efectúe 3 calibraciones diferentes del histograma sobre la imagen original, entre los valores 50-100, 40-120, 35-166. Visualice los resultados.

```
In [ ]: # para la calibración del histograma, el parámetro in_range se establece con los valores que queremos
rescaled_img_quito = exposure.rescale_intensity(quito_image, in_range=(50,100), out_range=(0,255)).astype(np.uint8)

# visualizamos la imagen original al lado de la imagen con expansión del contraste
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.imshow(quito_image, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen QUITO original")

plt.subplot(1,2,2)
plt.imshow(rescaled_img_quito, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen QUITO con calibración del histograma entre 50-100")
plt.show()

# visualizamos el histograma de la imagen resultado
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
histogram_original = calculate_histogram(quito_image, hist_title="Histograma QUITO original", show=False)
plt.subplot(1,2,2)
histogram_rescaled = calculate_histogram(rescaled_img_quito, hist_title="Histograma de la imagen QUITO con calibración")
plt.show()
```

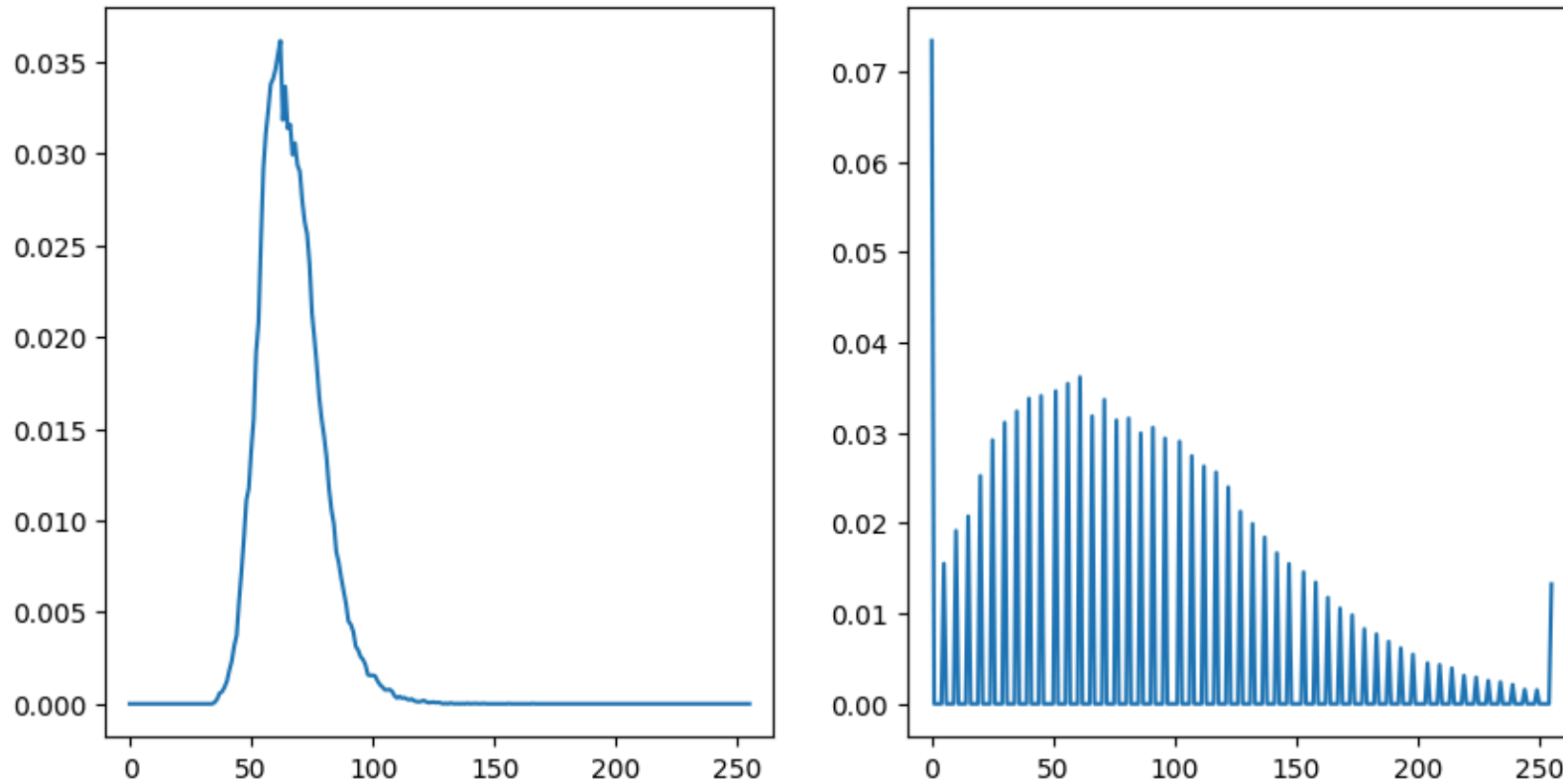
Imagen QUITO original



Imagen QUITO con calibración del histograma entre 50-100



Histograma QUITO originalHistograma de la imagen QUITO con calibración del histograma entre 50-100



```
In [ ]: rescaled_img_quito = exposure.rescale_intensity(quito_image, in_range=(40,120), out_range=(0,255)).astype(np.uint8)

# visualizamos la imagen original al lado de la imagen con expansión del contraste
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.imshow(quito_image, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen QUITO original")

plt.subplot(1,2,2)
plt.imshow(rescaled_img_quito, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen QUITO con calibración del histograma entre 40-120")
plt.show()

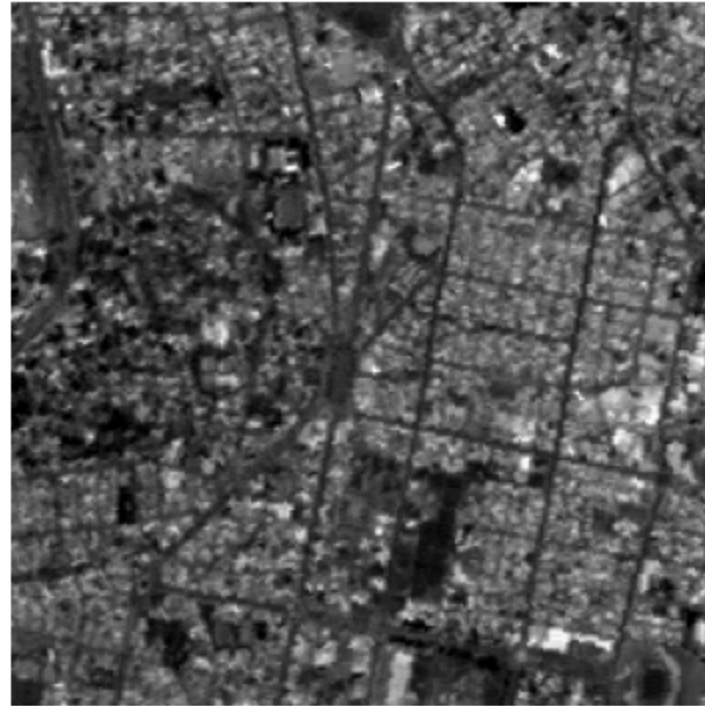
# visualizamos el histograma de la imagen resultado
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
histogram_original = calculate_histogram(quito_image, hist_title="Histograma QUITO original", show=False)
```

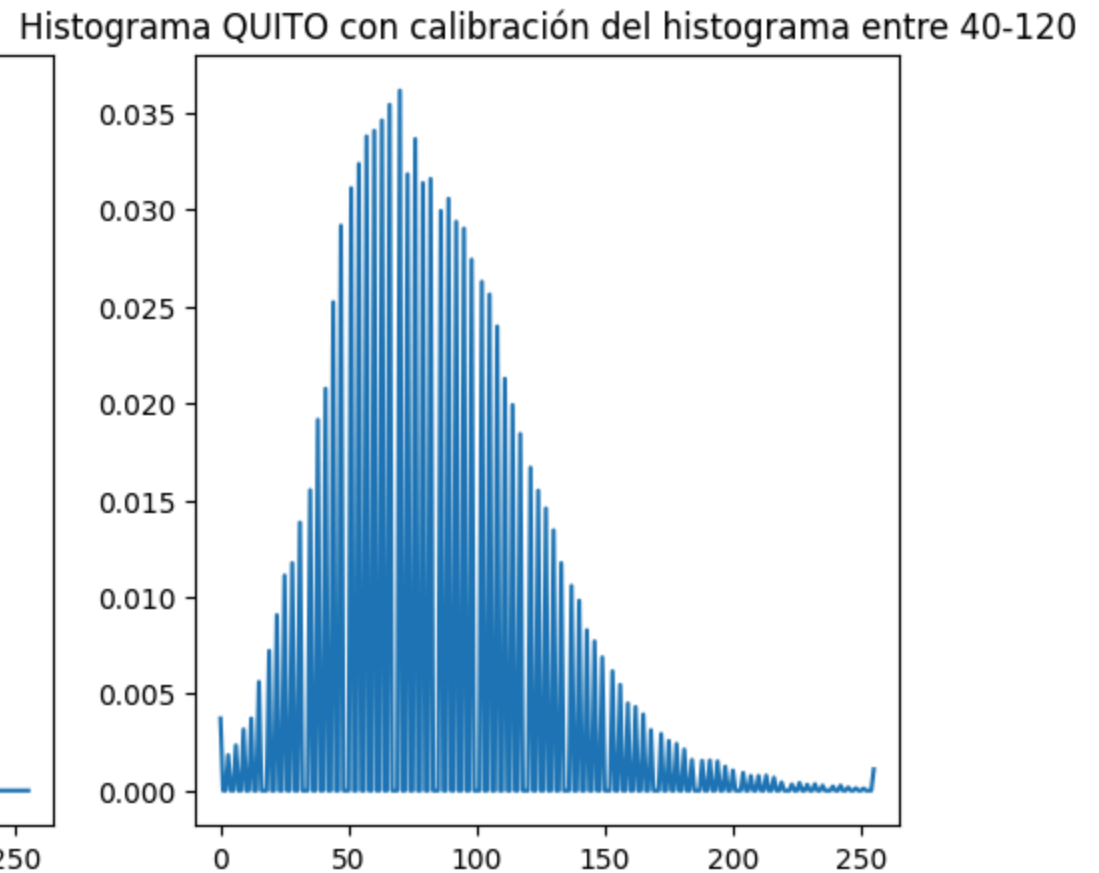
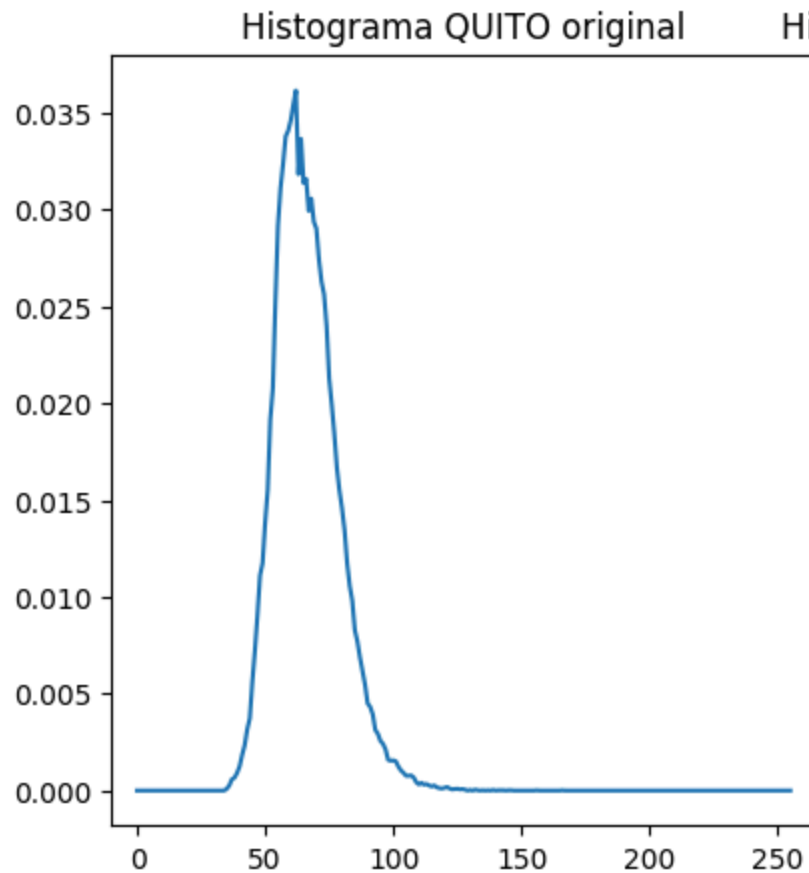
```
plt.subplot(1,2,2)  
histogram_rescaled = calculate_histogram(rescaled_img_quito, hist_title="Histograma QUITO con calibración del histograma"  
plt.show()
```

Imagen QUITO original



Imagen QUITO con calibración del histograma entre 40-120





```
In [ ]: rescaled_img_quito = exposure.rescale_intensity(quito_image, in_range=(35,166), out_range=(0,255)).astype(np.uint8)

# visualizamos la imagen original al lado de la imagen con expansión del contraste
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.imshow(quito_image, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen QUITO original")

plt.subplot(1,2,2)
plt.imshow(rescaled_img_quito, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen QUITO con calibración del histograma entre 35-166")
plt.show()

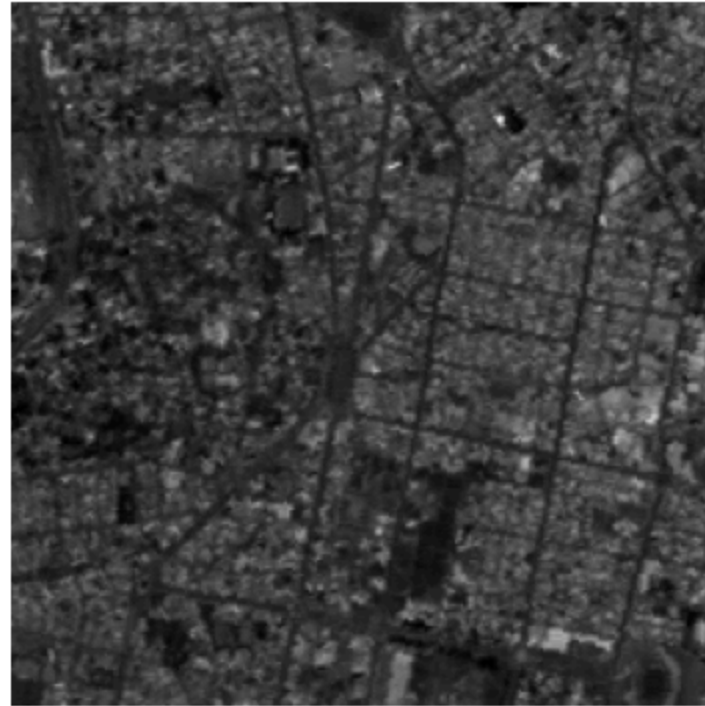
# visualizamos el histograma de la imagen resultado
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
```

```
histogram_original = calculate_histogram(quito_image, hist_title="Histograma QUITO original", show=False)
plt.subplot(1,2,2)
histogram_rescaled = calculate_histogram(rescaled_img_quito, hist_title="Histograma de la imagen QUITO con calibración")
plt.show()
```

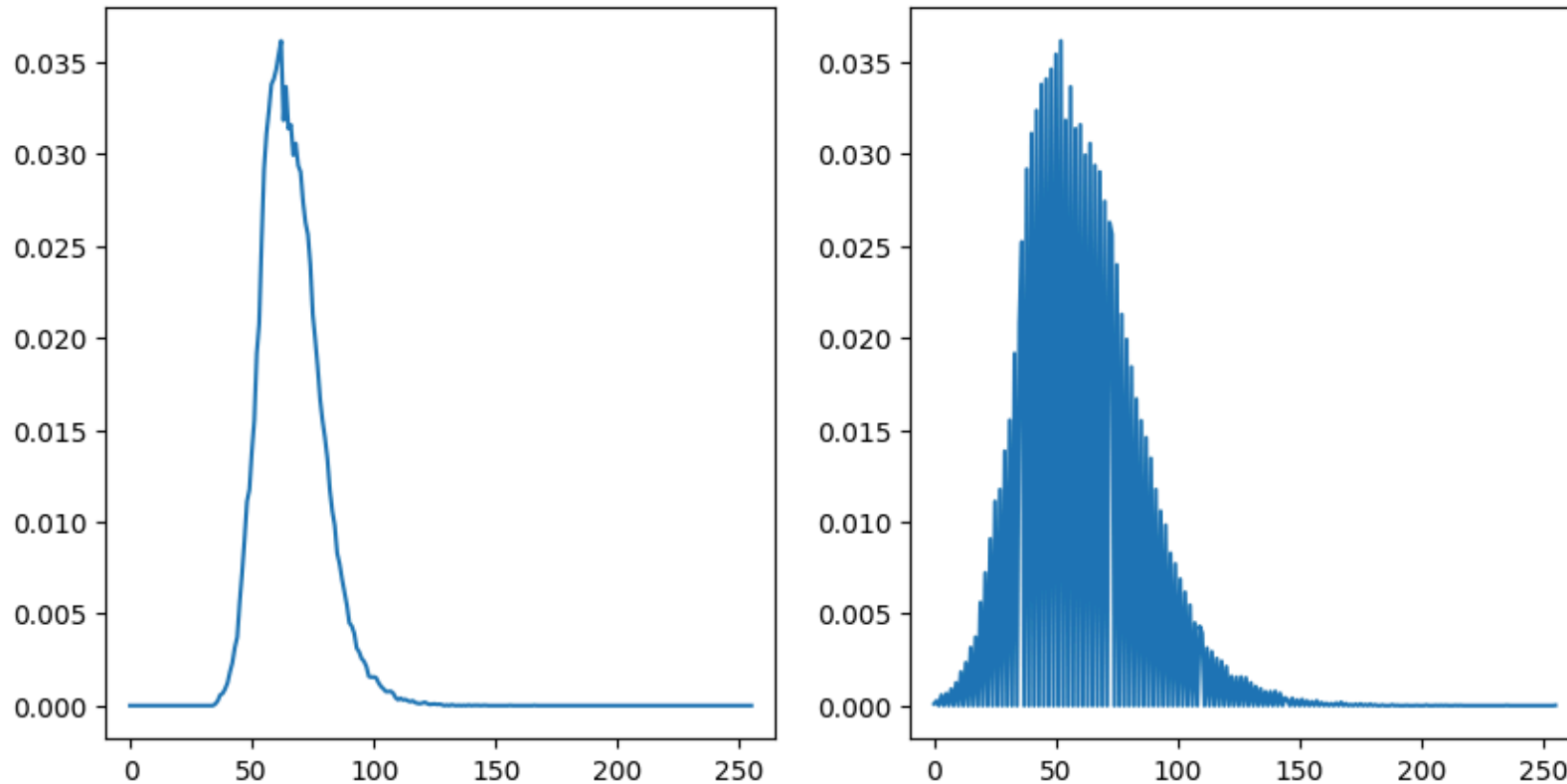
Imagen QUITO original



Imagen QUITO con calibración del histograma entre 35-166



Histograma QUITO original # histograma de la imagen QUITO con calibración del histograma entre 35-166



1.7. ¿Cuál es la diferencia entre estas imágenes? ¿Cuál presenta el mejor contraste? ¿Cuál permite ver mejor los detalles? ¿Por qué la calibración entre 35 y 166 da el mismo resultado que la expansión del contraste efectuada en el punto 4? ¿Conclusión?

Al modificar los rangos de calibración del histograma se observa que el nivel de contraste de la imagen depende directamente de dichos rangos. Un rango más estrecho (50 a 100) provoca una saturación más agresiva de los valores por debajo de 50 y por encima de 100, siendo esta expansión la que muestra el mejor contraste entre las tres opciones, puesto que, se incrementan las diferencias entre intensidades. Sin embargo, esta expansión resulta en una pérdida de detalles en las áreas más oscuras de la imagen. En este sentido, la que permite ver mejor los detalles como calles, parques y edificaciones es la calibración entre 40 y 120, ya que, aunque el contraste es menor que en el caso anterior se preservan más detalles en las áreas oscuras de la imagen.

Por otra parte, el rango más grande (35 a 166) da el mismo resultado que la expansión del contraste del punto 4 porque estos son exactamente los valores mínimos y máximos de tonalidades en la imagen original, indicando que 35 se mapea a 0 y 166 se mapea a 255,

resultado que es visible al obtenerse histogramas idénticos.

En conclusión, la calibración del histograma permite controlar de manera precisa el nivel de contraste y la preservación de detalles en la imagen. La elección del intervalo de intensidades es crítica, puesto que, rangos muy estrechos pueden exagerar el contraste a costa de perder información, mientras que, rangos más amplios producen mejoras más suaves y estables. Por ello, la calibración debe seleccionarse en función del contenido de la imagen y del objetivo del procesamiento.

2. Ecualización del histograma

2.1. Cree una nueva sección en su notebook.

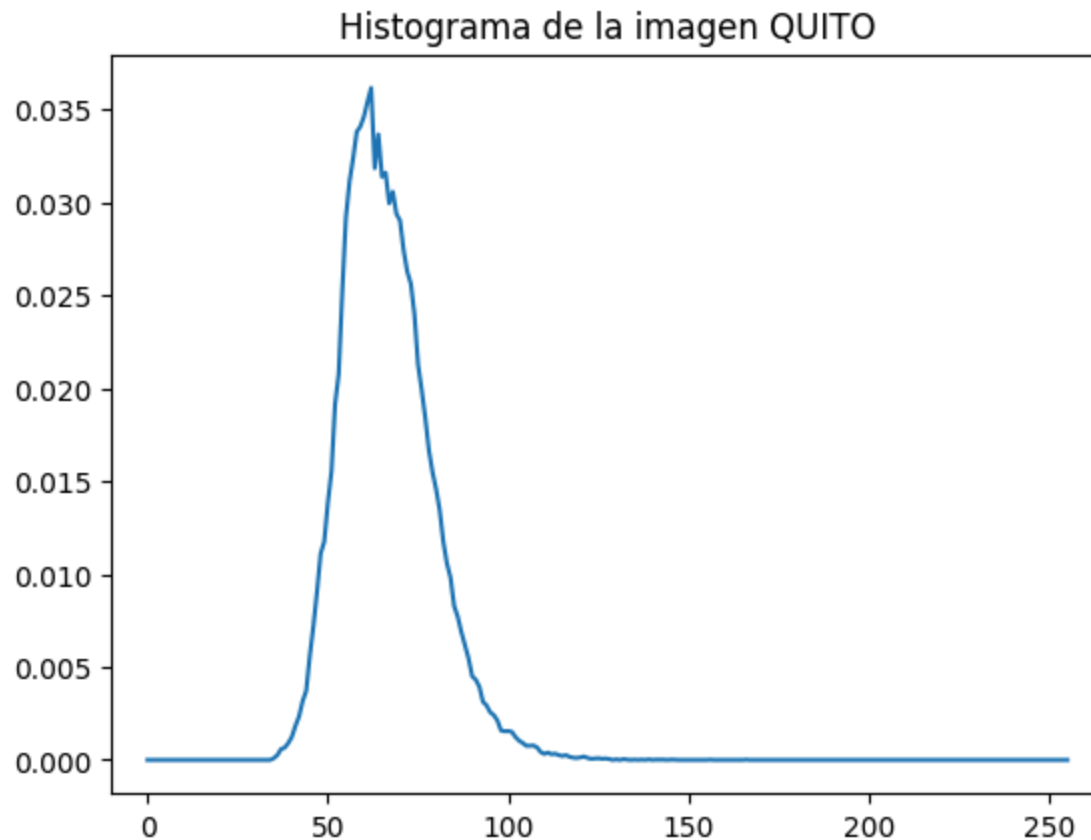
2.2. Cargue la imagen quito.png y visualicela.

```
In [ ]: quito_image = read_and_show_image_gray_scale(path + 'QUITO.png')
```



2.3. Visualice su histograma.

```
In [ ]: histogram_quito = calculate_histogram(quito_image, hist_title="Histograma de la imagen QUITO")
```



2.4. Efectúe una ecualización del histograma, visualice la imagen resultado y su histograma. En este caso particular, ¿a qué se debe la discontinuidad del histograma ecualizado?

```
In [ ]: # ecualizar el histograma de la imagen
imagen_equ = cv2.equalizeHist(quito_image)

# visualizamos la imagen original al lado de la imagen con ecualización del histograma
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.imshow(quito_image, cmap='gray', vmin=0, vmax=255)
```

```
plt.axis('off')
plt.title("Imagen QUITO original")

plt.subplot(1,2,2)
plt.imshow(imagen_equ, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen QUITO con ecualización del histograma")
plt.show()

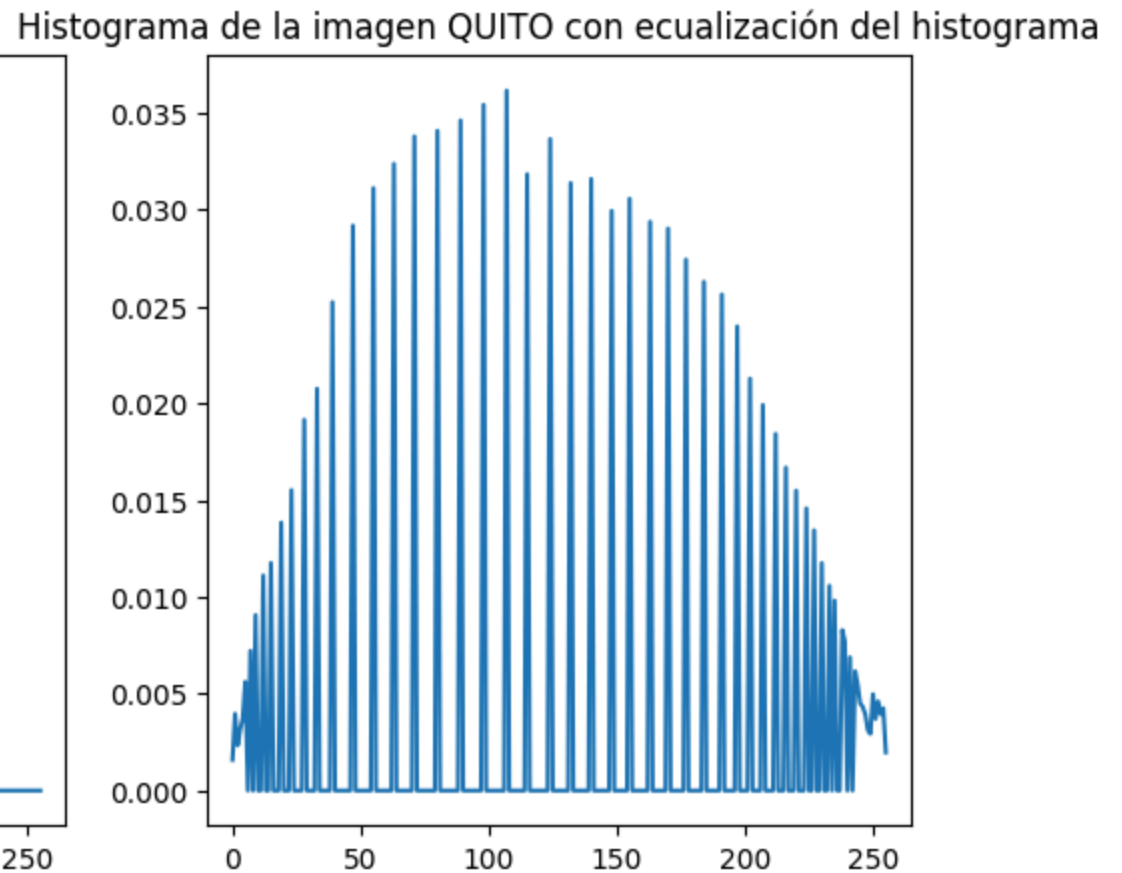
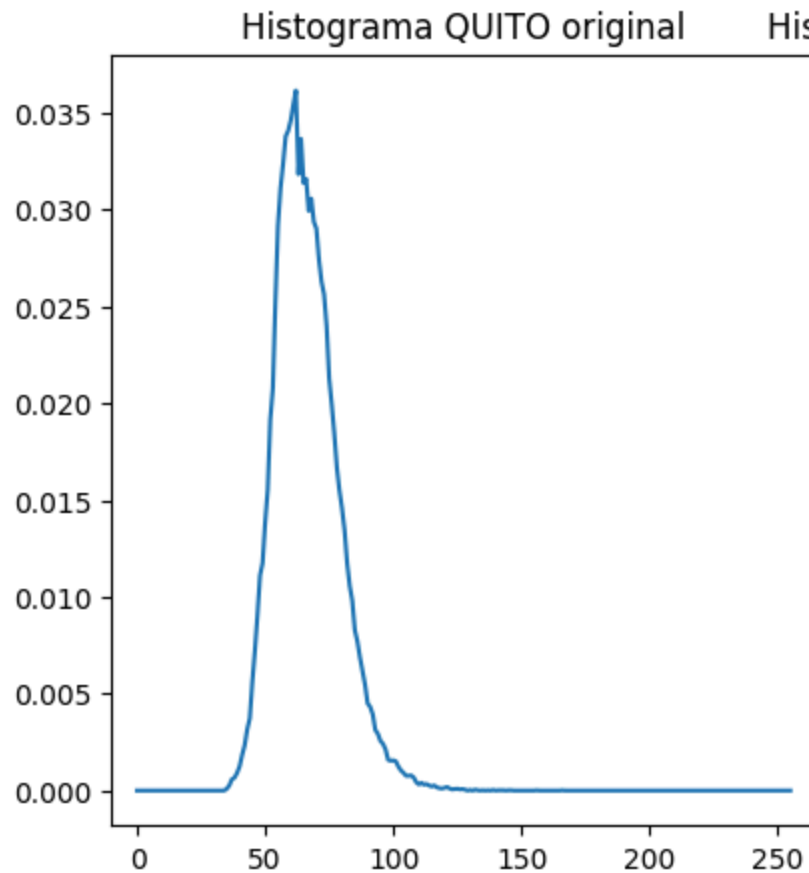
# visualizamos el histograma de la imagen resultado
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
histogram_original = calculate_histogram(quito_image, hist_title="Histograma QUITO original", show=False)
plt.subplot(1,2,2)
histogram_equ = calculate_histogram(imagen_equ, hist_title="Histograma de la imagen QUITO con ecualización del histograma")
plt.show()
```

Imagen QUITO original



Imagen QUITO con ecualización del histograma





La ecualización del histograma consiste en aplicar una función de transformación monótona creciente a los niveles de gris de la imagen de modo que aproximadamente se tenga la misma probabilidad de aparición para cada nivel de gris. A pesar de que se usa una función monótona creciente, es necesario discretizar los niveles de gris para que tomen valores enteros entre 0 y 255, por lo que, en el histograma ecualizado se puede observar discontinuidades o saltos. Esta operación produce una mejora significativa del contraste de la imagen, haciendo que los detalles sean más visibles.

2.5. Efectúe una segunda vez esta operación. es decir, aplique una ecualización del histograma a la imagen resultado del punto anterior. ¿Qué pasa? ¿Por qué?

```
In [ ]: # ecualizar el histograma de la imagen
imagen_equ = cv2.equalizeHist(quito_image)
imagen_equ2 = cv2.equalizeHist(imagen_equ)

# visualizamos la imagen original al lado de la imagen con ecualización del histograma y la imagen con ecualización del
```

```

plt.figure(figsize=(15,5))
plt.subplot(1,3,1)
plt.imshow(quito_image, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen QUITO original")

plt.subplot(1,3,2)
plt.imshow(imagen_equ, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen QUITO ecualizada")

plt.subplot(1,3,3)
plt.imshow(imagen_equ2, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen QUITO ecualizada dos veces")
plt.show()

# visualizamos el histograma de la imagen resultado
plt.figure(figsize=(15,5))
plt.subplot(1,3,1)
histogram_quito = calculate_histogram(quito_image, hist_title="Histograma de la imagen QUITO original", show=False)
plt.subplot(1,3,2)
histogram_equ = calculate_histogram(imagen_equ, hist_title="Histograma de QUITO ecualizada", show=False)
plt.subplot(1,3,3)
histogram_equ2 = calculate_histogram(imagen_equ2, hist_title="Histograma de QUITO ecualizada dos veces", show=False)
plt.show()

```

Imagen QUITO original

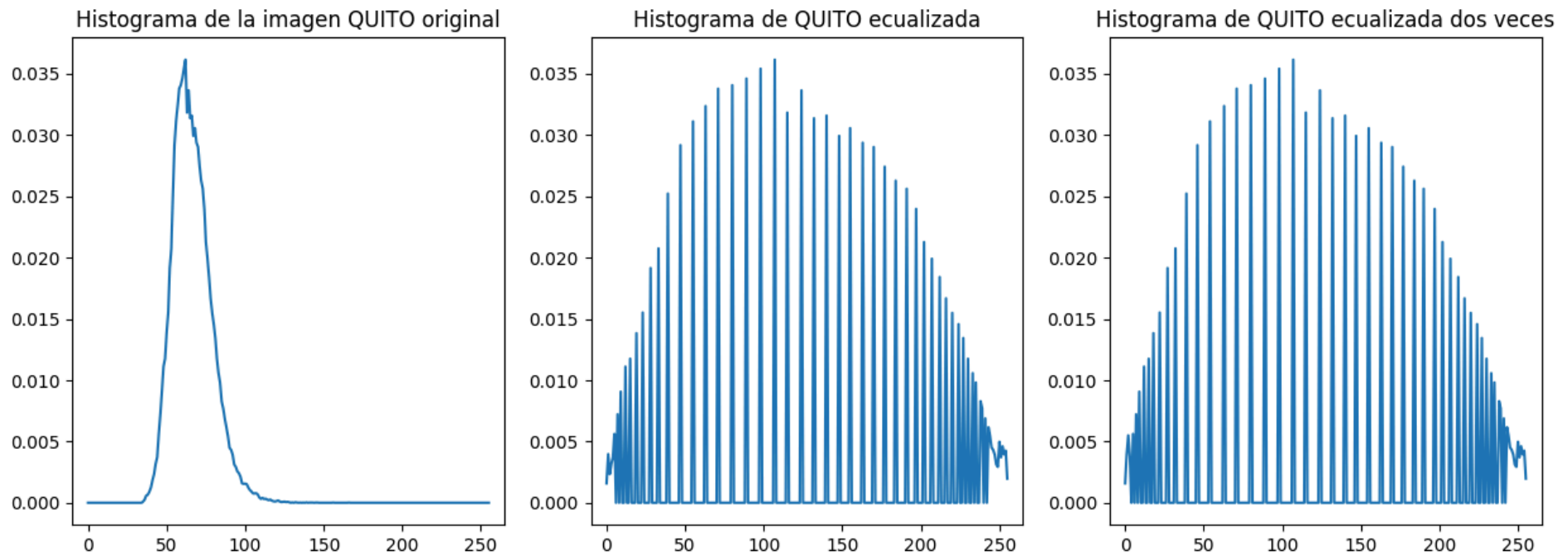


Imagen QUITO ecualizada



Imagen QUITO ecualizada dos veces





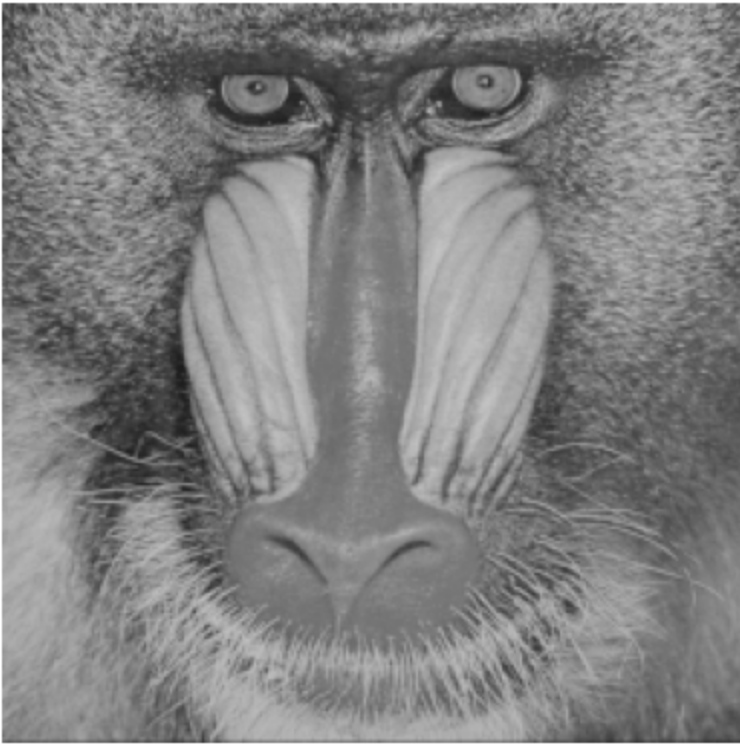
Podemos observar que al aplicar una segunda ecualización del histograma a la imagen obtenida del punto anterior, no se producen cambios significativos en la nueva imagen ni en su histograma. Esto se debe a que la primera ecualización ya ha distribuido los niveles de gris de manera uniforme, por lo que aplicar una segunda ecualización no tiene un efecto adicional sobre la imagen.

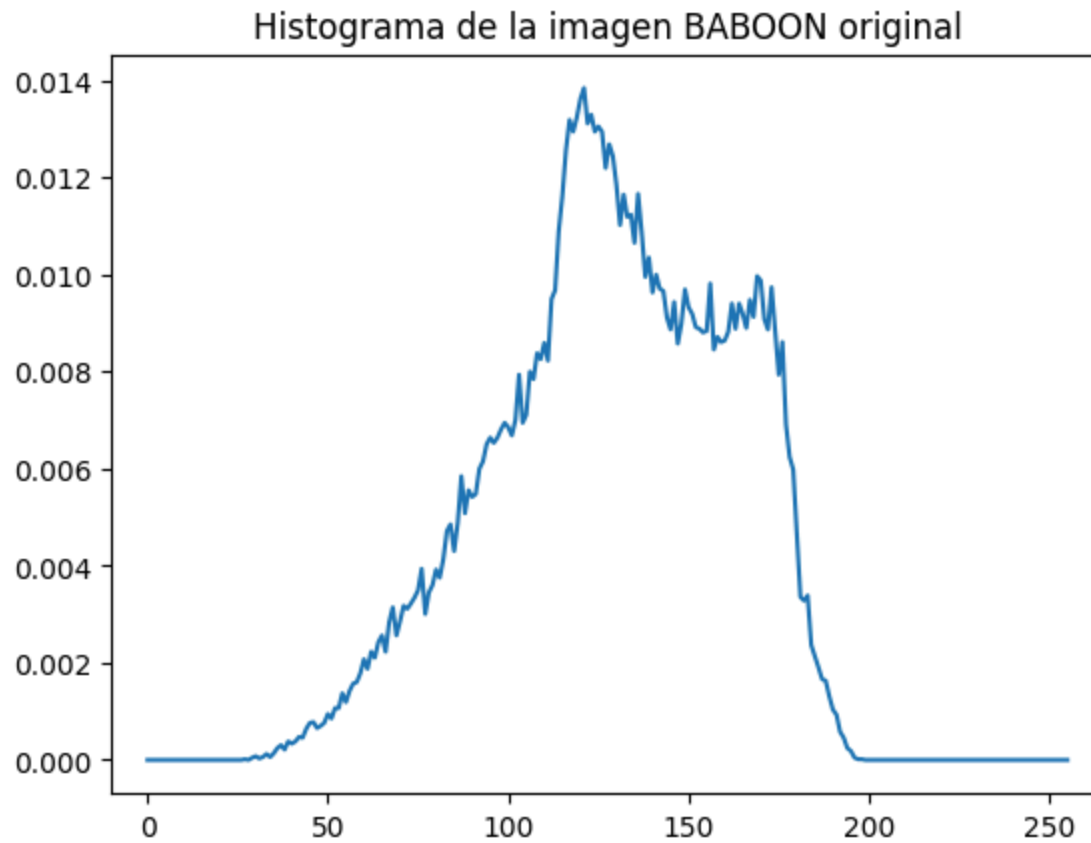
Nota: Se observa un ligero cambio en los valores cercanos a intensidad cero, el cual, puede deberse a aproximaciones en cálculos internos de la función.

2.6. Repita las operaciones de 1 a 4 con la imagen baboon.png. ¿Cómo es el resultado de esta transformación comparado con la ecualización de la imagen precedente (quito.png)? ¿Por qué?

```
In [ ]: baboon_image = read_and_show_image_gray_scale(path + 'BABOON.png')
baboon_histogram = calculate_histogram(baboon_image, hist_title="Histograma de la imagen BABOON original")

print(f"Valor de gris mínimo en la imagen BABOON original: {min(baboon_image.flatten())}")
print(f"Valor de gris máximo en la imagen BABOON original: {max(baboon_image.flatten())}")
```





Valor de gris mínimo en la imagen BABOON original: 27

Valor de gris máximo en la imagen BABOON original: 198

```
In [ ]: # ecualizar el histograma de la imagen
imagen_equ = cv2.equalizeHist(baboon_image)
imagen_equ2 = cv2.equalizeHist(imagen_equ)

# visualizamos la imagen original al lado de la imagen con ecualización del histograma y la imagen con ecualización del
plt.figure(figsize=(15,5))
plt.subplot(1,3,1)
plt.imshow(baboon_image, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen BABOON original")

plt.subplot(1,3,2)
plt.imshow(imagen_equ, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen BABOON ecualizada")

plt.subplot(1,3,3)
```



```
plt.imshow(imagen_equ2, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen BABOON ecualizada dos veces")
plt.show()

# visualizamos el histograma de la imagen resultado
plt.figure(figsize=(15,5))
plt.subplot(1,3,1)
histogram_baboon = calculate_histogram(baboon_image, hist_title="Histograma de la imagen BABOON original", show=False)
plt.subplot(1,3,2)
histogram_equ = calculate_histogram(imagen_equ, hist_title="Histograma de BABOON ecualizada", show=False)
plt.subplot(1,3,3)
histogram_equ2 = calculate_histogram(imagen_equ2, hist_title="Histograma de BABOON ecualizada dos veces", show=False)
plt.show()
```

Imagen BABOON original

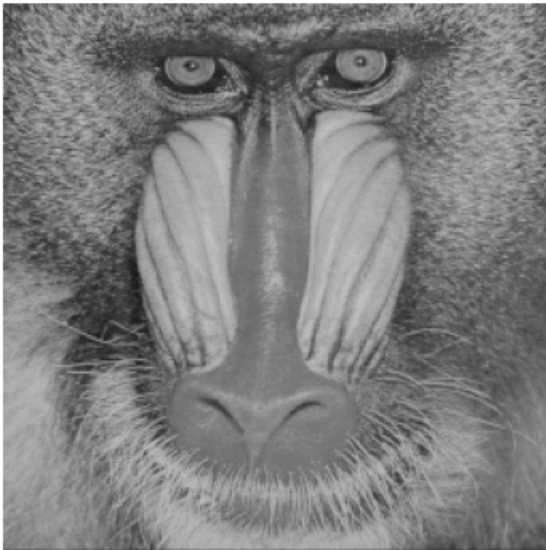


Imagen BABOON ecualizada

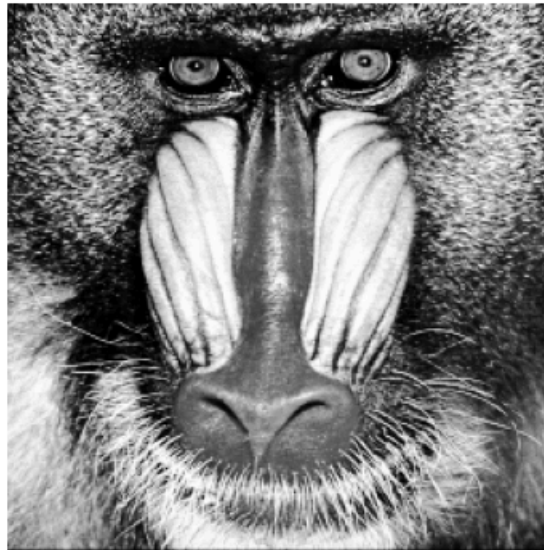
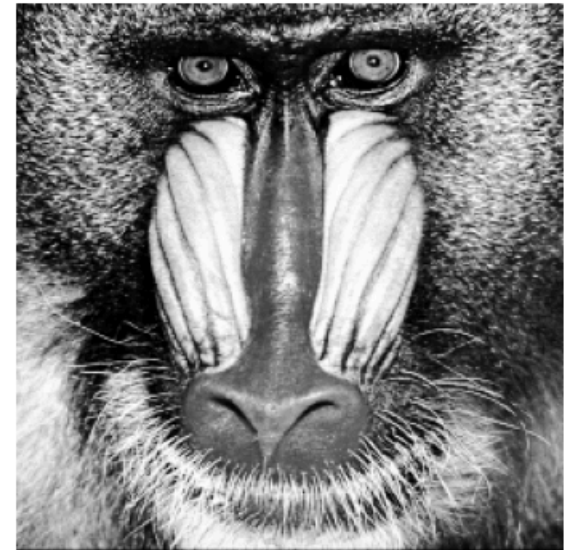
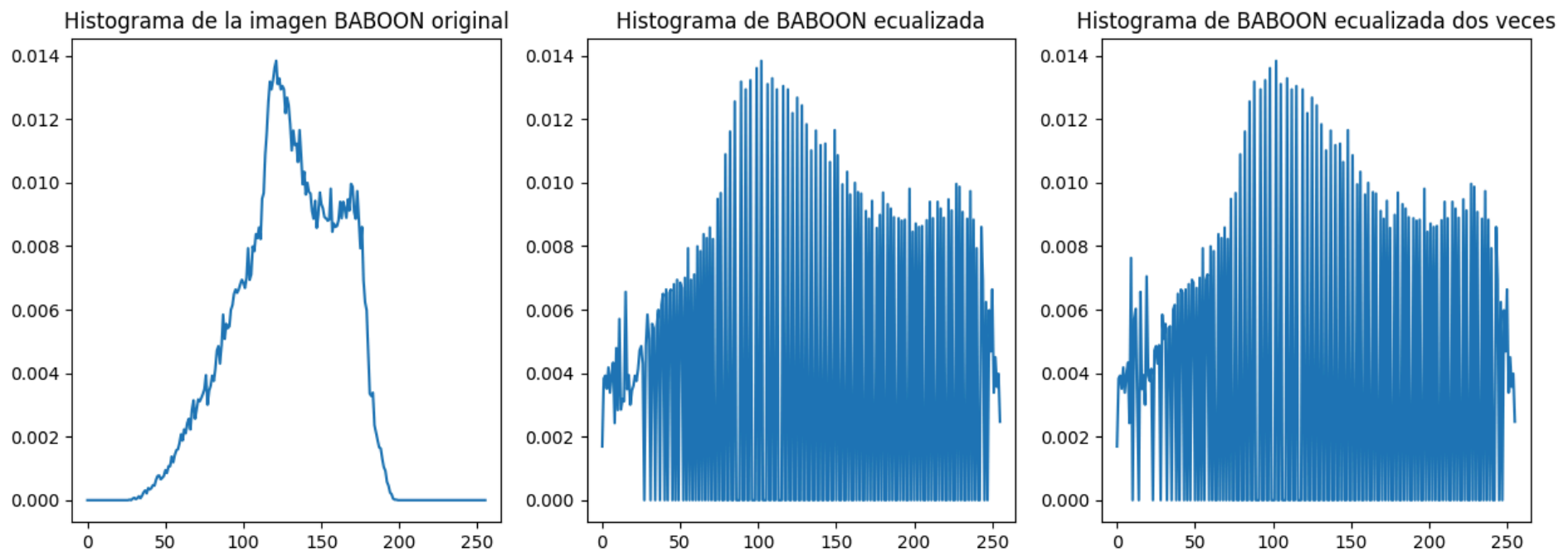


Imagen BABOON ecualizada dos veces





La imagen BABOON presenta un rango más amplio de niveles de gris, aproximadamente entre 27 y 198, con una mayor concentración en valores intermedios. En general, muestra una distribución de tonalidades más dispersa que la imagen QUITO, lo que indica una mayor variabilidad de intensidades desde el inicio.

La ecualización del histograma en esta imagen mejora el contraste y permite resaltar con mayor claridad los detalles en las facciones del babuino, el cual, presenta variaciones fuertes de textura (pelaje) y bordes bien definidos. En los histogramas se observa una distribución más uniforme de los niveles de gris en comparación con la imagen QUITO, con discontinuidades menos pronunciadas, aunque todavía presentes debido a la discretización del proceso. También, se puede percibir un realce excesivo de la textura (el pelaje se vuelve más "granulado" o más agresivo visualmente), porque la ecualización amplifica diferencias locales que ya eran significativas.

Al aplicar la ecualización por segunda vez, se detectan ligeras variaciones en los niveles de gris más oscuros, aproximadamente entre 0 y 30. Sin embargo, estos cambios no se perciben como un gran cambio entre las imágenes de ambas ecualizaciones.

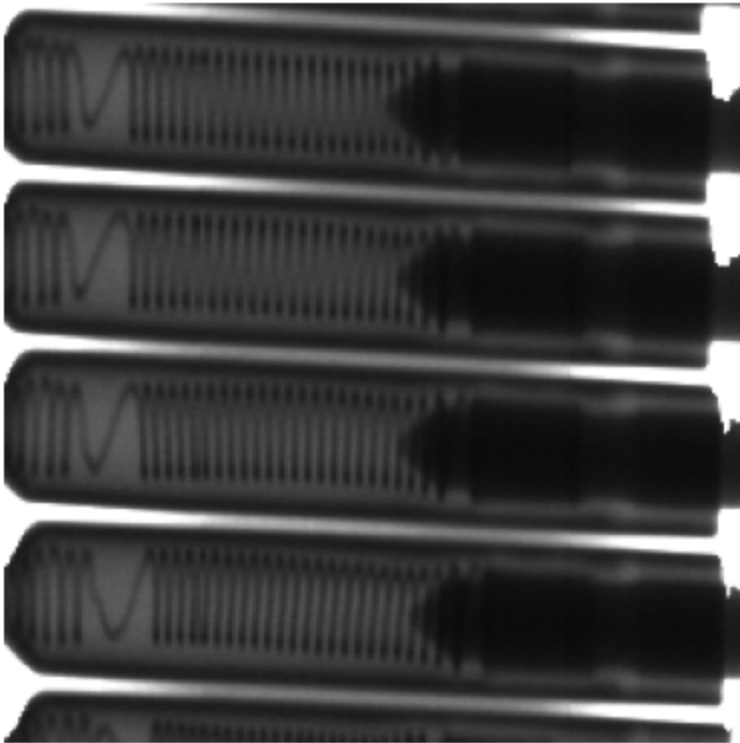
Al comparar ambas imágenes, se observa que el aumento de contraste en BABOON es menos drástico que en QUITO, lo cual, se debe a la mejor distribución inicial de los niveles de gris. No obstante, en ambos casos la ecualización del histograma contribuye significativamente a mejorar la visualización de detalles.

3. Comparación entre diferentes transformaciones del histograma

3.1. Cree una nueva sección en su notebook.

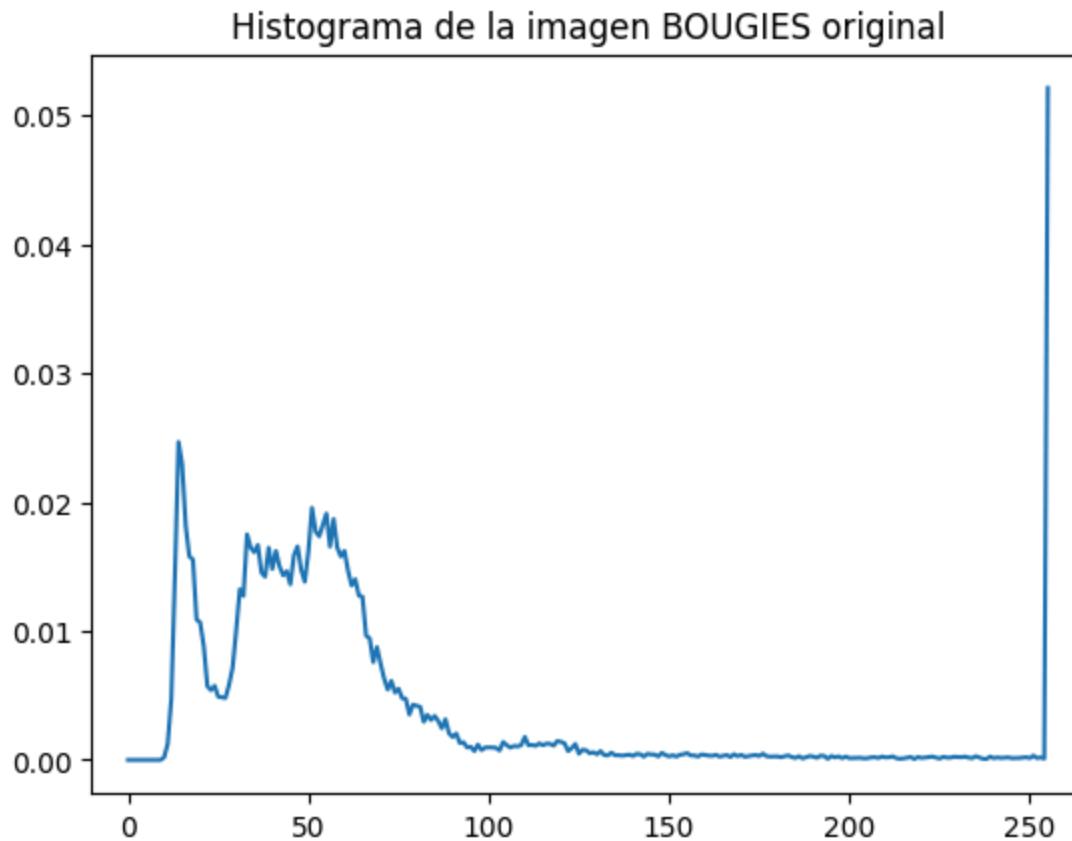
3.2. Cargue la imagen de trabajo y visualicela.

```
In [ ]: bougies_image = read_and_show_image_gray_scale(path + 'BOUGIES.png')
```



3.3. Visualice su histograma.

```
In [ ]: bougies_histogram = calculate_histogram(bougies_image, hist_title="Histograma de la imagen BOUGIES original", show=True)
```



3.4. Efectúe una expansión del contraste, visualice la imagen resultado y su histograma. Compare las imagenes (original y después de la expansión) junto con sus histogramas. ¿A qué se debe este resultado?

```
In [ ]: # expansión del contraste
rescaled_img_bougies = exposure.rescale_intensity(bougies_image, in_range=(bougies_image.flatten().min(),bougies_image.
print("Valores mínimos y máximos de BOUGIES:", (min(bougies_image.flatten()), max(bougies_image.flatten())))
# visualización de la imagen original al lado de la imagen con expansión del contraste
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.imshow(bougies_image, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen BOUGIES original")
plt.subplot(1,2,2)
plt.imshow(rescaled_img_bougies, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
```

```
plt.title("Imagen BOUGIES con expansión del contraste")
plt.show()

# histogramas
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
histogram_bougies = calculate_histogram(bougies_image, hist_title="Histograma de BOUGIES original", show=False)
plt.subplot(1,2,2)
histogram_rescaled_bougies = calculate_histogram(rescaled_img_bougies, hist_title="Histograma de BOUGIES con expansión", show=False)
plt.show()
```

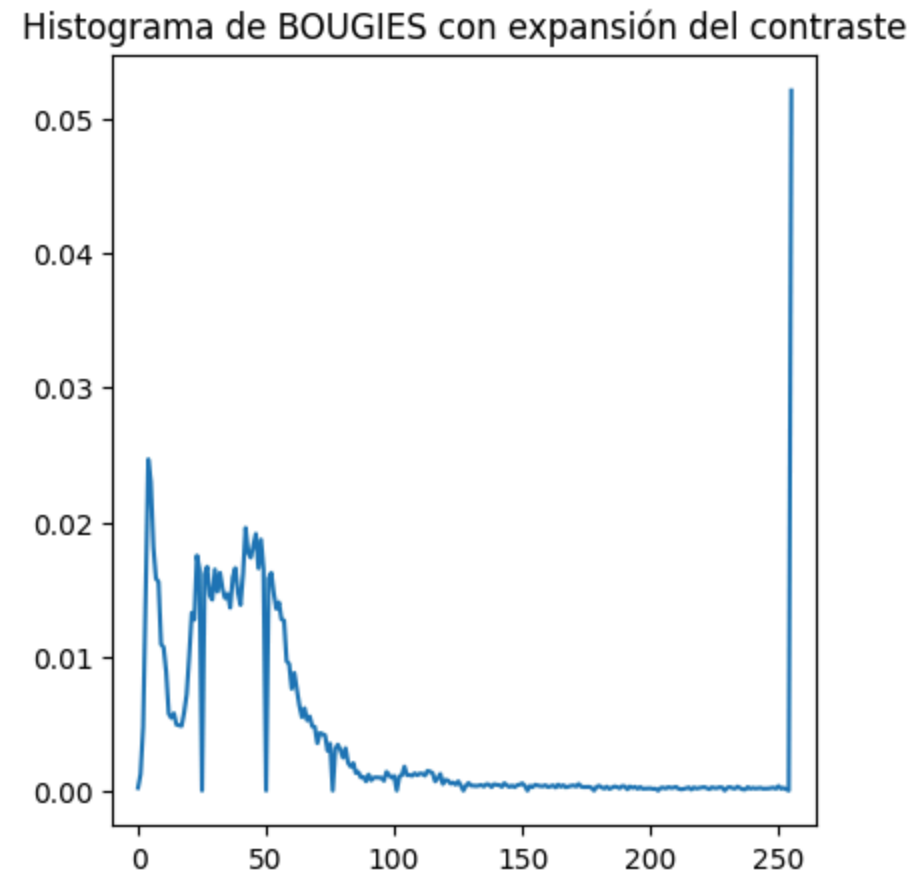
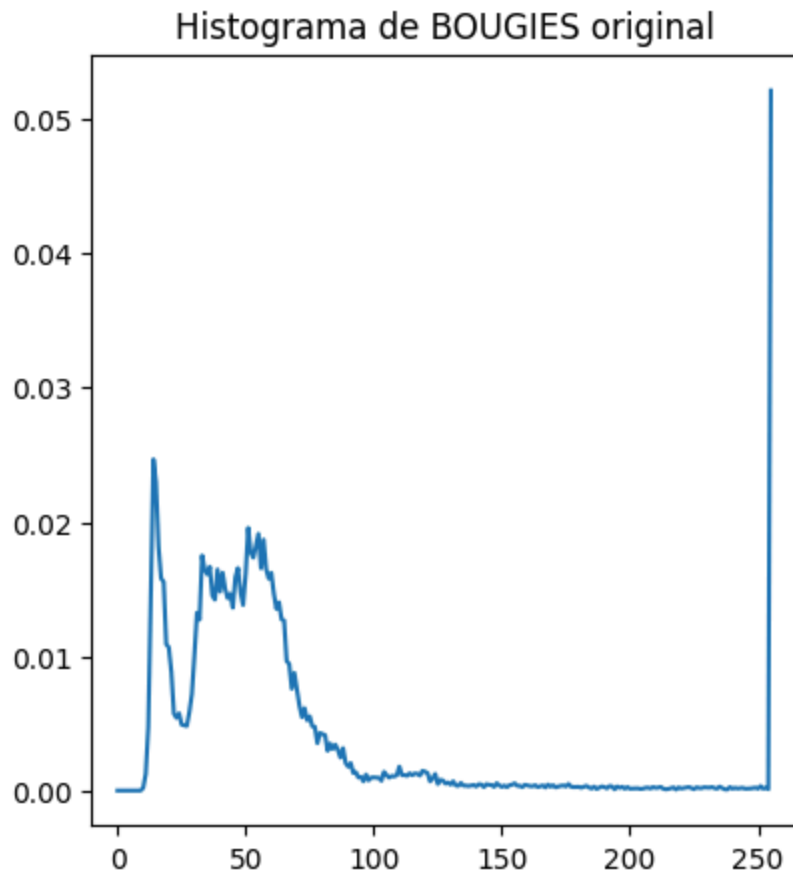
Valores mínimos y máximos de BOUGIES: (np.uint8(10), np.uint8(255))

Imagen BOUGIES original



Imagen BOUGIES con expansión del contraste





Entre la imagen original de BOUGIES y la imagen resultado de la expansión del contraste se observan que, se pierden algunos valores de tonalidades de grises (decaen acero) y se desplaza levemente la curva hacia la izquierda, provocando cambios mínimos en la imagen que no son fácilmente apreciables a simple vista. Esto se debe a que los valores mínimos y máximos de la imagen original son 10 y 255 respectivamente, indicando que la imagen ya tiene un rango casi completo de niveles de gris, por lo cual, su extensión a 0-255 no produce cambios significativos.

3.5. Sobre la imagen original efectúe ahora una ecualización del histograma, visualice la imagen resultado y su histograma. Compare las imágenes resultado de la expansión (punto anterior) y de la ecualización (resultado actual), junto con sus histogramas. Comente sus observaciones.

```
In [ ]: # expansión del contraste
rescaled_img_bougies = exposure.rescale_intensity(bougies_image, in_range=(bougies_image.flatten().min(),bougies_image.
```

```

# ecualización del histograma
equ_img_bougies = cv2.equalizeHist(bougies_image)

# visualización de la imagen original al lado de la imagen con expansión del contraste
plt.figure(figsize=(15,5))
plt.subplot(1,3,1)
plt.imshow(bougies_image, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen BOUGIES original")
plt.subplot(1,3,2)
plt.imshow(rescaled_img_bougies, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen BOUGIES con expansión del contraste")
plt.subplot(1,3,3)
plt.imshow(equ_img_bougies, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen BOUGIES con ecualización del histograma")
plt.show()

# histogramas
plt.figure(figsize=(15,5))
plt.subplot(1,3,1)
histogram_bougies = calculate_histogram(bougies_image, hist_title="Histograma de BOUGIES original", show=False)
plt.subplot(1,3,2)
histogram_rescaled_bougies = calculate_histogram(rescaled_img_bougies, hist_title="Histograma de BOUGIES con expansión")
plt.subplot(1,3,3)
histogram_equ_bougies = calculate_histogram(equ_img_bougies, hist_title="Histograma de BOUGIES con ecualización del his")
plt.show()

```

Imagen BOUGIES original



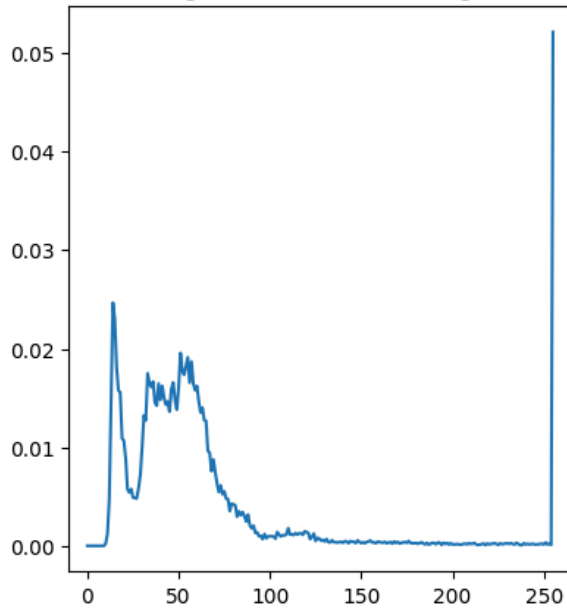
Imagen BOUGIES con expansión del contraste



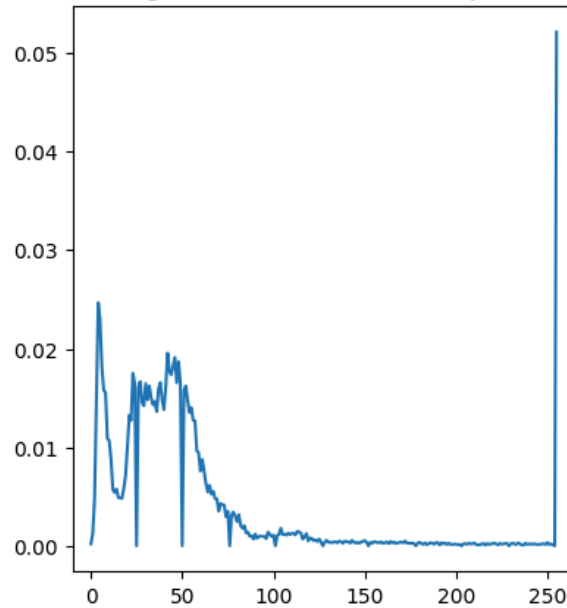
Imagen BOUGIES con ecualización del histograma



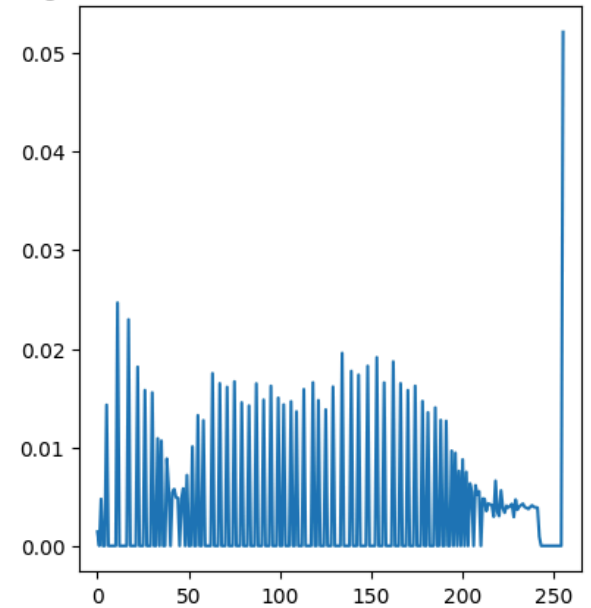
Histograma de BOUGIES original



Histograma de BOUGIES con expansión



Histograma de BOUGIES con ecualización del histograma



Comparando los resultados de la ecualización del histograma con la expansión del contraste, observamos que la ecualización sí produce cambios significativos en BOUGIES.png.

En la imagen original, hay un pico elevado de valores en blanco y el resto distribuido en niveles de grises bajos, entre 10 y 60 aproximadamente. Los demás valores (tonalidades de grises) no cuentan con una gran cantidad de píxeles.

Así mismo, la imagen con expansión de contraste no presenta variaciones significativas con respecto a la original, ya que, el rango de valores es aproximadamente igual al rango completo, conservando los 2 picos de distribución de píxeles en las diferentes tonalidades de grises.

Por el contrario, al ecualizar el histograma (función que busca una distribución uniforme de los niveles de gris), estos valores bajos se expanden a lo largo del rango completo, dando como resultado una imagen con mejor contraste y mayor distinción de detalles (el resorte dentro de la bujía se ve con mayor claridad).

3.6. Sobre la imagen original aplique ahora una transformación logarítmica y multiplique la imagen resultado por un valor constante de 46. ¿Por qué es necesario multiplicar la imagen de salida por un factor? Visualice la imagen resultado y su histograma. ¿Cómo es el histograma resultado?

```
In [ ]: # transformación logarítmica
# al tener valores en 255 en 8 bits, si sumo 1 van a pasar a ser 0. Convierto la imagen a 16 bits para evitarlo
bougies_image16 = bougies_image.astype(np.uint16)
log_transformed = (np.log(bougies_image16 + 1)) * 46

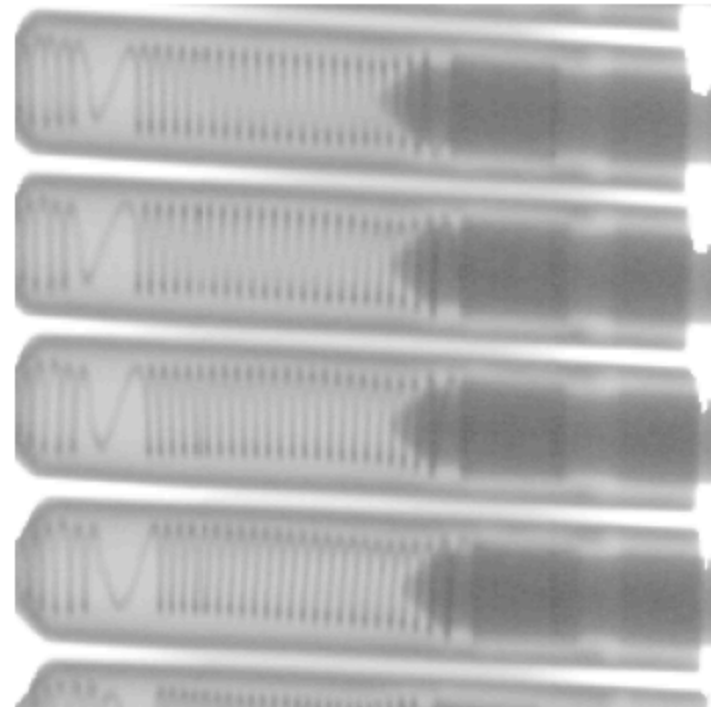
# visualización de la imagen original y la transformada logarítmica
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.imshow(bougies_image, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen BOUGIES original")
plt.subplot(1,2,2)
plt.imshow(log_transformed, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen BOUGIES con transformación logarítmica")
plt.show()

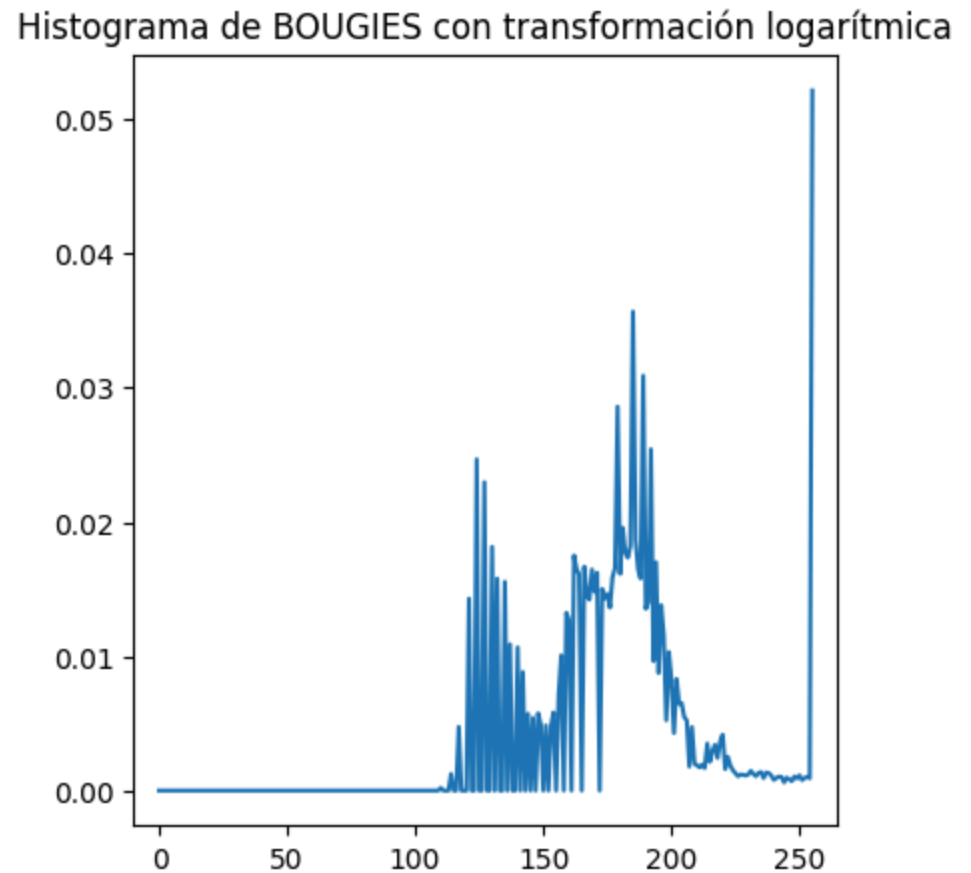
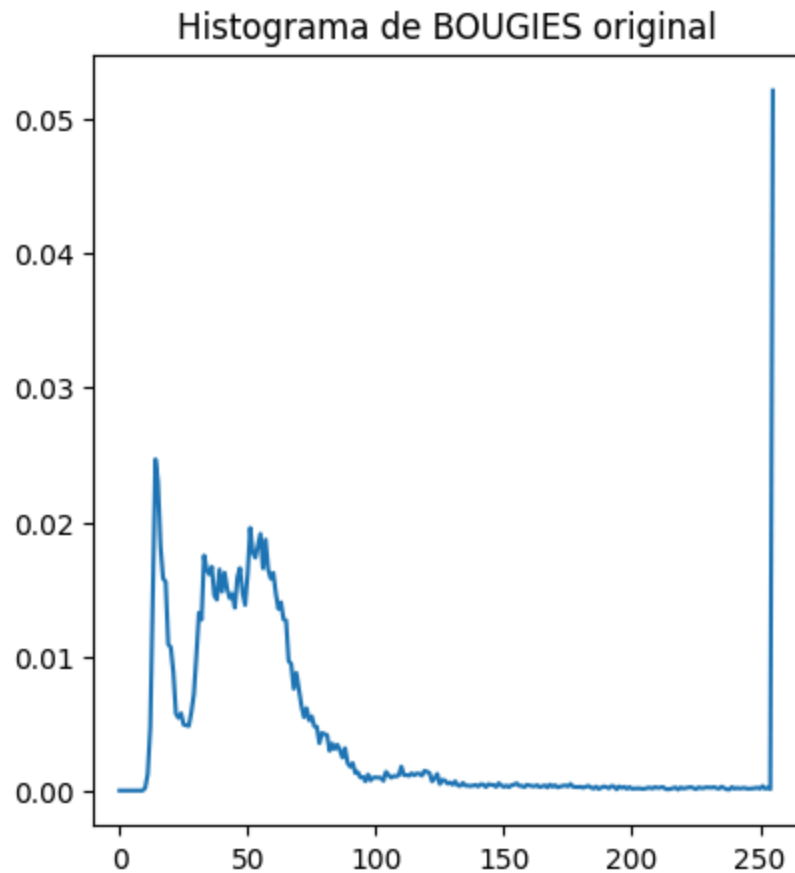
# visualizar histogramas
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
histogram_bougies = calculate_histogram(bougies_image, hist_title="Histograma de BOUGIES original", show=False)
plt.subplot(1,2,2)
histogram_log = calculate_histogram(log_transformed, hist_title="Histograma de BOUGIES con transformación logarítmica",
plt.show()
```


Imagen BOUGIES original



Imagen BOUGIES con transformación logarítmica





Al realizar la transformación logarítmica, los valores más bajos de gris se expanden, mientras que los valores más altos se comprimen, es decir, muestran una menor diferencia entre ellos, por ejemplo, el valor 251 se transforma con la aproximación a 52, al igual que 252, perdiendo una tonalidad.

El valor máximo al aplicar el logaritmo sería aproximadamente 5.545 , que al multiplicarlo por 46 se obtiene un valor cercano a 255. En este sentido, la multiplicación se hace para escalar los valores de forma que tome aproximadamente todo el rango de grises entre 0 y 255.

Esta transformación produce una mejora de contraste en las áreas más oscuras de la imagen, puesto que, se pasa de un rango inicial de 10-100 a uno más amplio(110-220). Por esta razón, al ser la imagen BOUGIES una imagen con un rango de grises bastante oscuro, la transformación logarítmica muestra de mejor manera los detalles en las áreas oscuras. En el histograma de resultado se observa entonces una expansión y traslación de los valores grises más oscuros a grises más claros, mientras que, los valores altos (claros) tienen una compresión.

3.7. Sobre la imagen original efectúe ahora una especificación de dos histogramas diferentes de entrada. Visualice la imagen resultado, los histogramas esperados y los histogramas obtenidos. Compare los resultados con los de los puntos anteriores. Comente sus observaciones.

```
In [ ]: bougies_image = bougies_image.astype(np.uint8)
#Generamos la distribución acumulativa del histograma
cdf = np.cumsum(histogram_bougies)
#Generamos un arreglo con valores aleatorios según la distribución
pixel_values1 = np.interp(np.random.rand(bougies_image.shape[0]*bougies_image.shape[1]), cdf, range(0,256))
#Convertimos el arreglo en una imagen.
i_to_match= pixel_values1.reshape(bougies_image.shape).astype(np.uint8)
matched_image = exposure.match_histograms(bougies_image, i_to_match)
# convertimos a uint8 y cortamos en el rango de 0 a 255
matched_image = np.clip(matched_image, 0, 255).astype(np.uint8)

# Generamos un segundo arreglo aleatorio con la misma distribución
pixel_values2 = np.interp(np.random.rand(bougies_image.shape[0]*bougies_image.shape[1]), cdf, range(0,256))
i_to_match2 = pixel_values2.reshape(bougies_image.shape).astype(np.uint8)
matched_image2 = exposure.match_histograms(bougies_image, i_to_match2)
matched_image2 = np.clip(matched_image2, 0, 255).astype(np.uint8)

# visualizacion de imagenes (original, especificación 1 y especificación 2)
plt.figure(figsize=(15,5))
plt.subplot(1,3,1)
plt.imshow(bougies_image, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen BOUGIES original")
plt.subplot(1,3,2)
plt.imshow(i_to_match, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen con distribución aleatoria 1")
plt.subplot(1,3,3)
plt.imshow(matched_image, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen BOUGIES con especificación de histograma 1")
plt.show()

# visualizacion de histogramas
plt.figure(figsize=(15,5))
plt.subplot(1,3,1)
histogram_bougies = calculate_histogram(bougies_image, hist_title="Histograma de BOUGIES original", show=False)
```

```

plt.subplot(1,3,2)
histogram_random1 = calculate_histogram(i_to_match, hist_title="Histograma de la imagen con distribución aleatoria 1",
plt.subplot(1,3,3)
histogram_matched1 = calculate_histogram(matched_image, hist_title="Histograma de BOUGIES con especificación de histogr
plt.show()

# Mismo procedimiento para la segunda imagen de especificación
plt.figure(figsize=(15,5))
plt.subplot(1,3,1)
plt.imshow(bougies_image, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen BOUGIES original")
plt.subplot(1,3,2)
plt.imshow(i_to_match2, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen con distribución aleatoria 2")
plt.subplot(1,3,3)
plt.imshow(matched_image2, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen BOUGIES con especificación de histograma 2")
plt.show()

# visualizacion de histogramas
plt.figure(figsize=(15,5))
plt.subplot(1,3,1)
histogram_bougies = calculate_histogram(bougies_image, hist_title="Histograma de BOUGIES original", show=False)
plt.subplot(1,3,2)
histogram_random2 = calculate_histogram(i_to_match2, hist_title="Histograma de imagen con distribución aleatoria 2", sh
plt.subplot(1,3,3)
histogram_matched2 = calculate_histogram(matched_image2, hist_title="Histograma de BOUGIES con especificación de histog
plt.show()

```

Imagen BOUGIES original



Imagen con distribución aleatoria 1

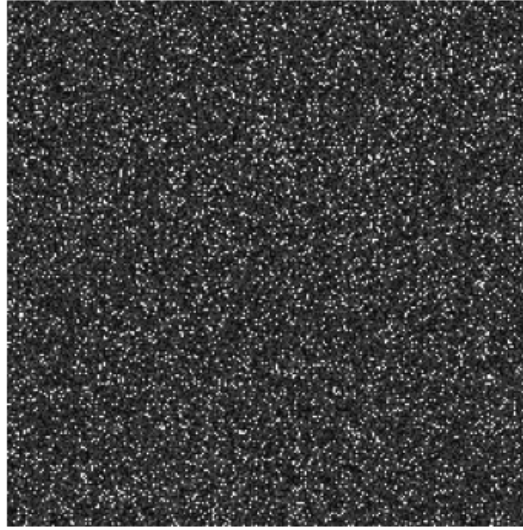
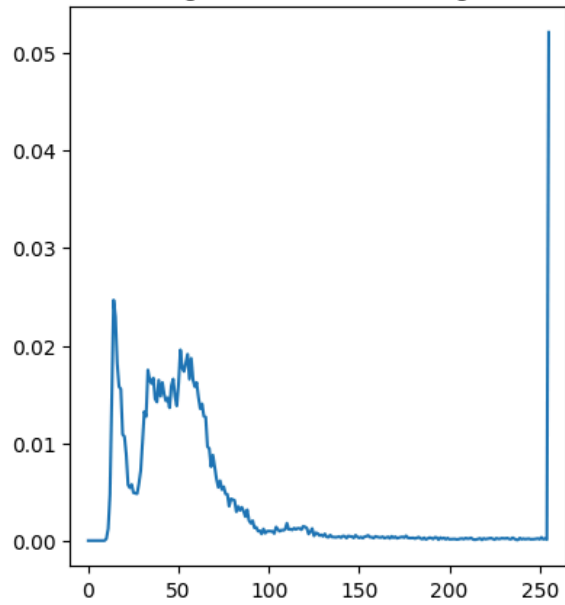


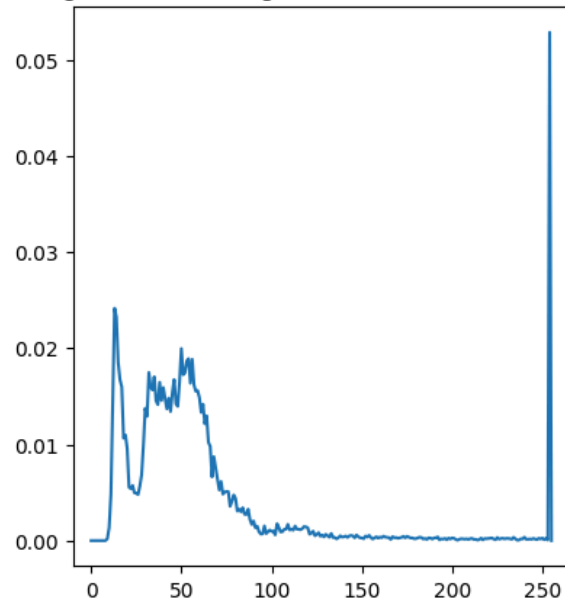
Imagen BOUGIES con especificación de histograma 1



Histograma de BOUGIES original



Histograma de la imagen con distribución aleatoria 1



Histograma de BOUGIES con especificación de histograma 1

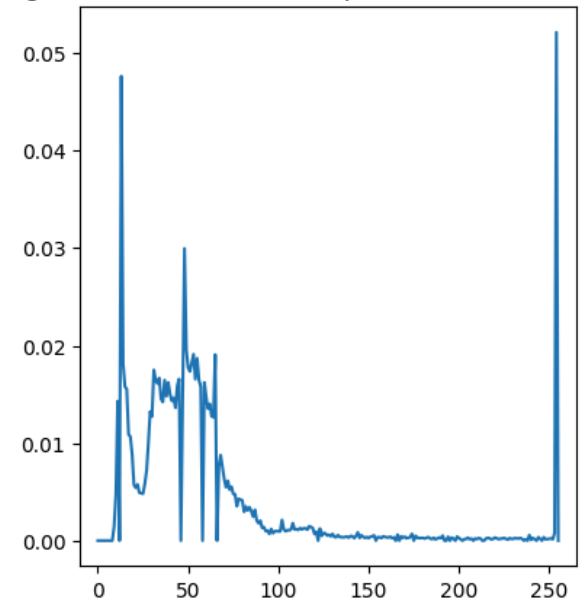


Imagen BOUGIES original



Imagen con distribución aleatoria 2

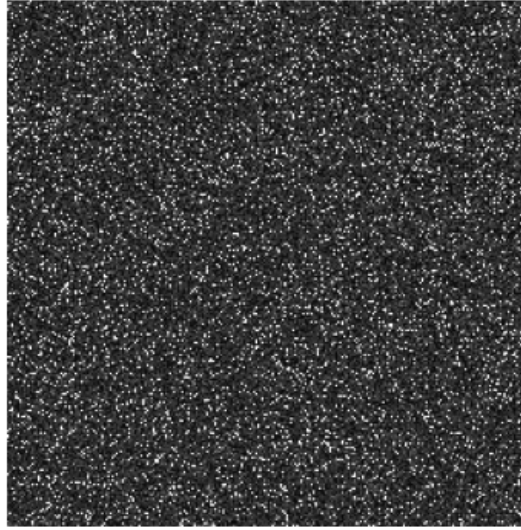
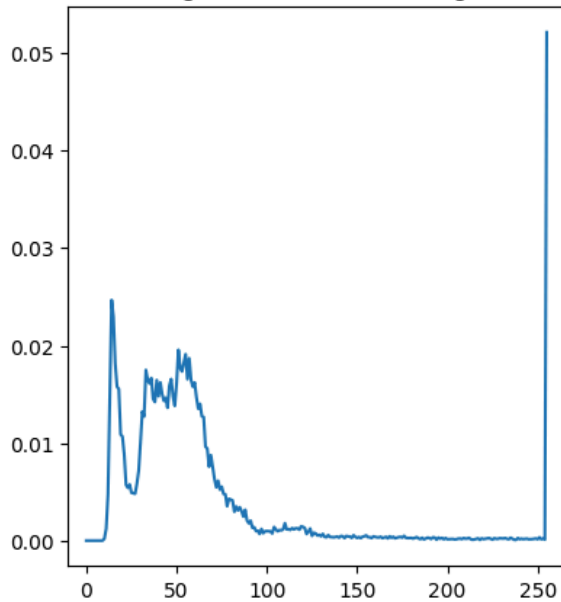


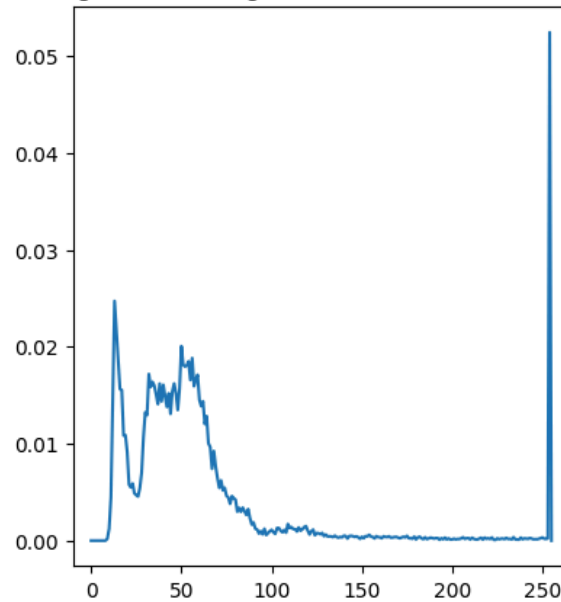
Imagen BOUGIES con especificación de histograma 2



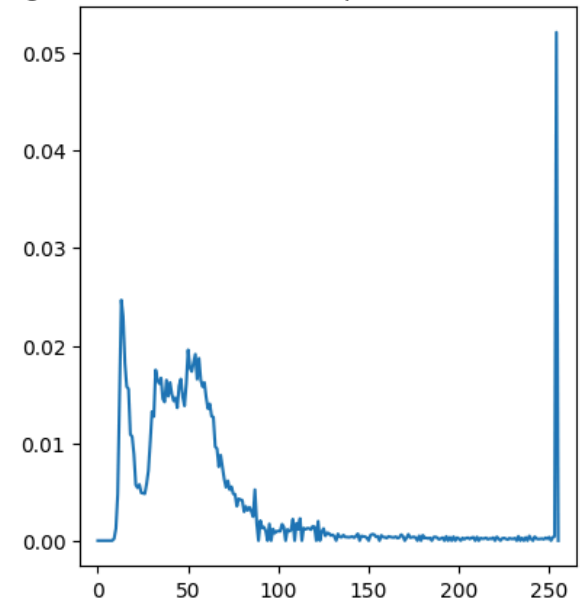
Histograma de BOUGIES original



Histograma de imagen con distribución aleatoria 2



Histograma de BOUGIES con especificación de histograma 2



El proceso de especificación del histograma ajusta la distribución de intensidades de manera que las tonalidades de la imagen original se asemejen a las de una imagen de referencia. En este caso, al usar la distribución de la imagen BOUGIES como referencia, la imagen aleatoria muestra una distribución de niveles de gris muy similar a la imagen original, presentando cambios más significativos en torno a los niveles de grises más altos (ceranos a 255). En consecuencia, al hacer match con esta imagen, el resultado no cambia mucho con respecto a la imagen original. La efectividad de esta técnica depende entonces de la imagen de referencia utilizada, pues si llegan a tener distribuciones parecidas, y no necesariamente imágenes parecidas (como se observa en la imagen aleatoria), los cambios no serán percibidos.

3.8. Haga una comparación entre los tipos de transformaciones ¿Qué impactos tiene cada una? ¿Cómo pueden ser usadas en el ámbito de mejoramiento de la calidad de la imagen?

Cada tipo de transformación del histograma tiene impactos diferentes en la imagen; la expansión del histograma mejora el contraste de imágenes cuando su distribución de valores de grises se encuentran concentradas en un rango estrecho; la ecualización por su parte ofrece una mejora significativa del contraste al distribuir los niveles de grises de manera uniforme, pero presenta poco control sobre el resultado final; la transformación logarítmica es especialmente útil para mejorar el contraste en áreas oscuras de la imagen, pero puede resultar en una pérdida de detalles en áreas claras; Por último, la especificación del histograma permite un control más preciso sobre el resultado y una mayor capacidad de adaptación al problema, en contraparte, es necesario elegir de forma adecuada la imagen de referencia para obtener resultados satisfactorios.

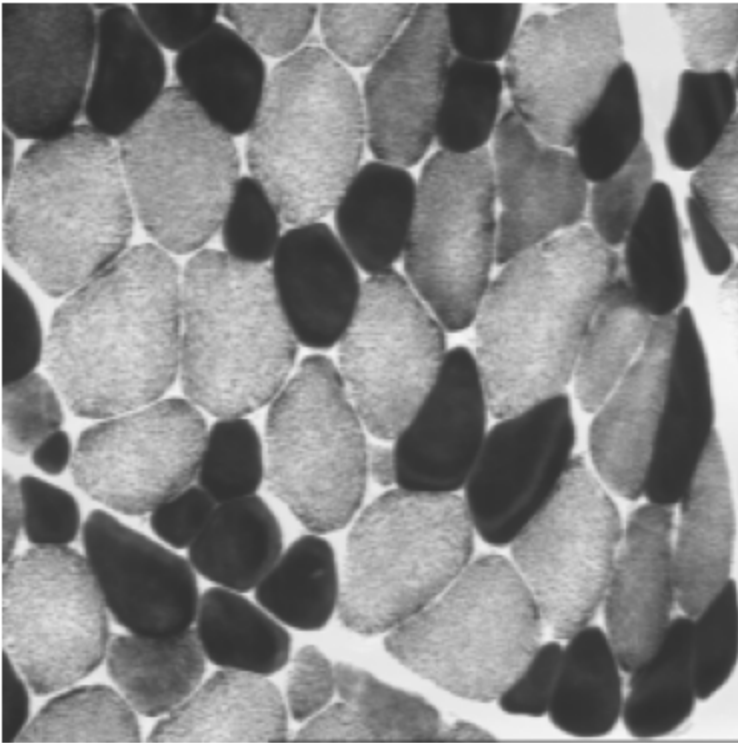
En la práctica, la elección de la transformación depende del objetivo del procesamiento y del tipo de contraste que se desea resaltar, ya que todas permiten mejorar la calidad de la imagen, pero con distintos niveles de control y efectos visuales.

4. Umbralización simple

4.1. Cree una nueva sección en su notebook.

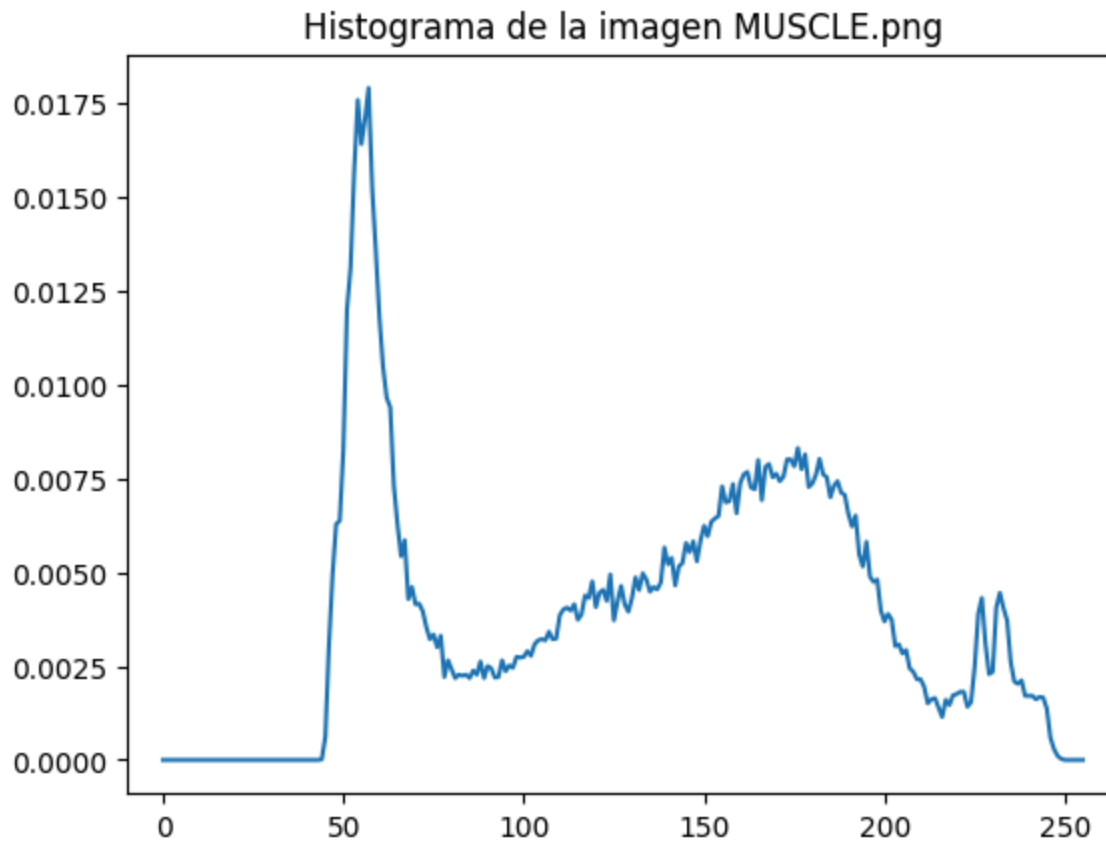
4.2. Cargue la imagen de trabajo ****muscle.png** y visualícela.**

```
In [ ]: muscle_image = read_and_show_image_gray_scale(path + 'MUSCLE.png')
```

4.3. Visualice su histograma y escoja un valor S de nivel de gris que permita separar aproximadamente las fibras oscuras del resto de la imagen. ¿Cuál es este valor?

```
In [ ]: muscle_histogram = calculate_histogram(muscle_image, hist_title="Histograma de la imagen MUSCLE.png")
```

Las fibras oscuras de la imagen se concentran en el pico de valores bajos del histograma, entre 40 y 90 aproximadamente, por lo que, un valor de S adecuado puede ser 85, separando el pico de las fibras oscuras del resto de la imagen.

4.4. Efectúe una calibración del histograma entre los valores S y $S+1$ de la imagen. Visualice la imagen resultado y su histograma ¿Cuál es su conclusión?

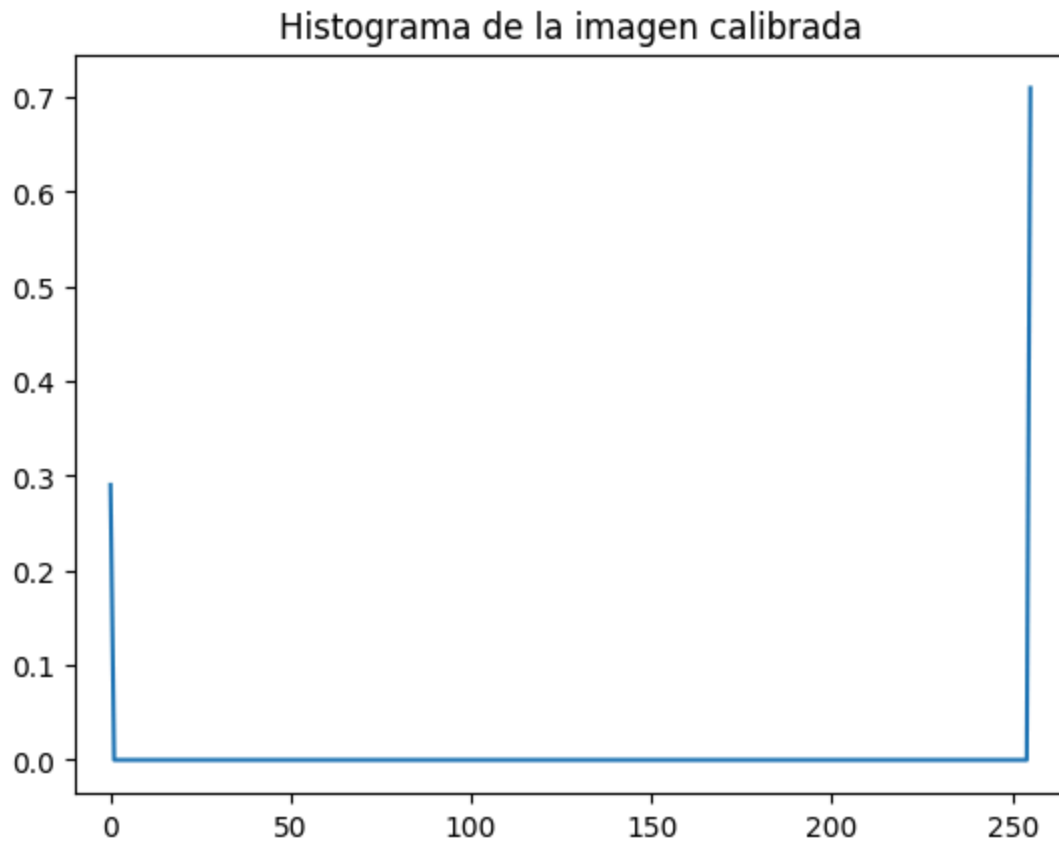
```
In [ ]: rescaled_img = exposure.rescale_intensity(muscle_image, in_range=(85,86), out_range=(0,255)).astype(np.uint8)

# visualizamos la imagen resultante
plt.imshow(rescaled_img, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Calibración del histograma entre 85 y 86")
plt.show()

# calculamos y mostramos el histograma de la imagen resultante
rescaled_histogram = calculate_histogram(rescaled_img, hist_title="Histograma de la imagen calibrada")
```

Calibración del histograma entre 85 y 86





La calibración del histograma entre los valores S y $S+1$, es decir entre 85 y 86 está mapeando todos los valores menores a 85 a 0, y todos los valores mayores a 86 a 255, sin tomar ningún otro valor puesto a que no hay valores entre 85 y 86, porque los valores de las imágenes son discretos. Esto hace que la imagen resultado sea una imagen binaria, en donde las fibras oscuras se vean completamente negras en un fondo blanco. Por consiguiente, en el histograma se observa únicamente dos niveles de gris (0 y 255).

4.5. Repita la misma operación utilizando el módulo de umbralización simple, con el umbral S . ¿Conclusión?

```
In [ ]: thres_value, thres_image = cv2.threshold(muscle_image, 85, 255, cv2.THRESH_BINARY)

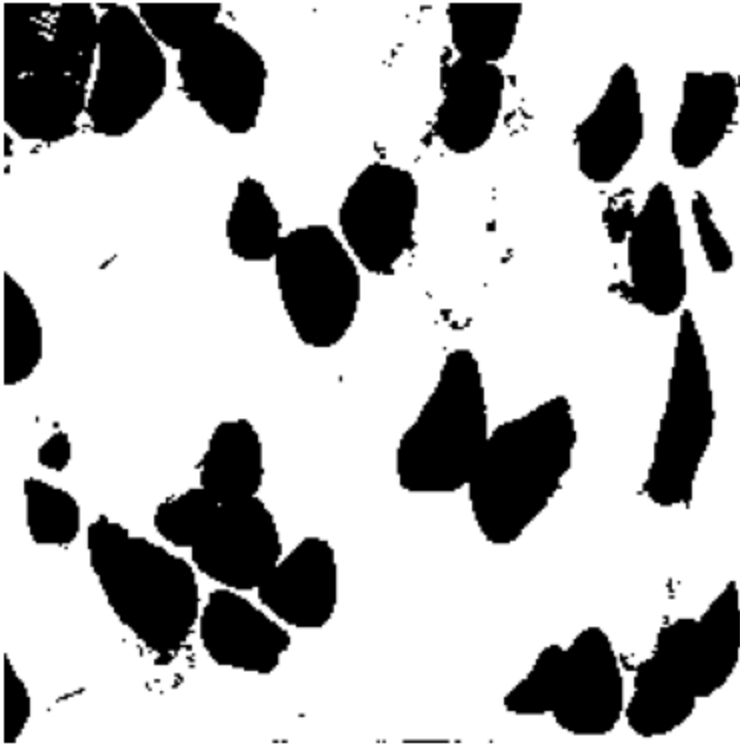
print(f"Valor de umbral utilizado: {thres_value}")

# visualizamos la imagen resultante
plt.imshow(thres_image, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
```

```
plt.title("Imagen umbralizada con umbral 85")
plt.show()
```

Valor de umbral utilizado: 85.0

Imagen umbralizada con umbral 85



Al realizar una umbralización simple con el umbral S se obtiene exactamente el mismo resultado que con la calibración del histograma entre S y $S+1$. Esto es debido a que, la umbralización simple es una técnica de segmentación que asigna el valor 0 a los píxeles con valor menor o igual a S y 1 a los píxeles con valor mayor a S .

4.6. ¿Qué resultado dan los métodos de umbralización automática basados sobre la varianza? ¿Cómo eligen estos métodos el umbral a aplicar? Visualice las imágenes resultado y sus histogramas.

```
In [ ]: # umbralización automática con método Otsu
otsu_value, otsu_image = cv2.threshold(muscle_image, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

print(f"Valor de umbral automático (Otsu): {otsu_value}")
```

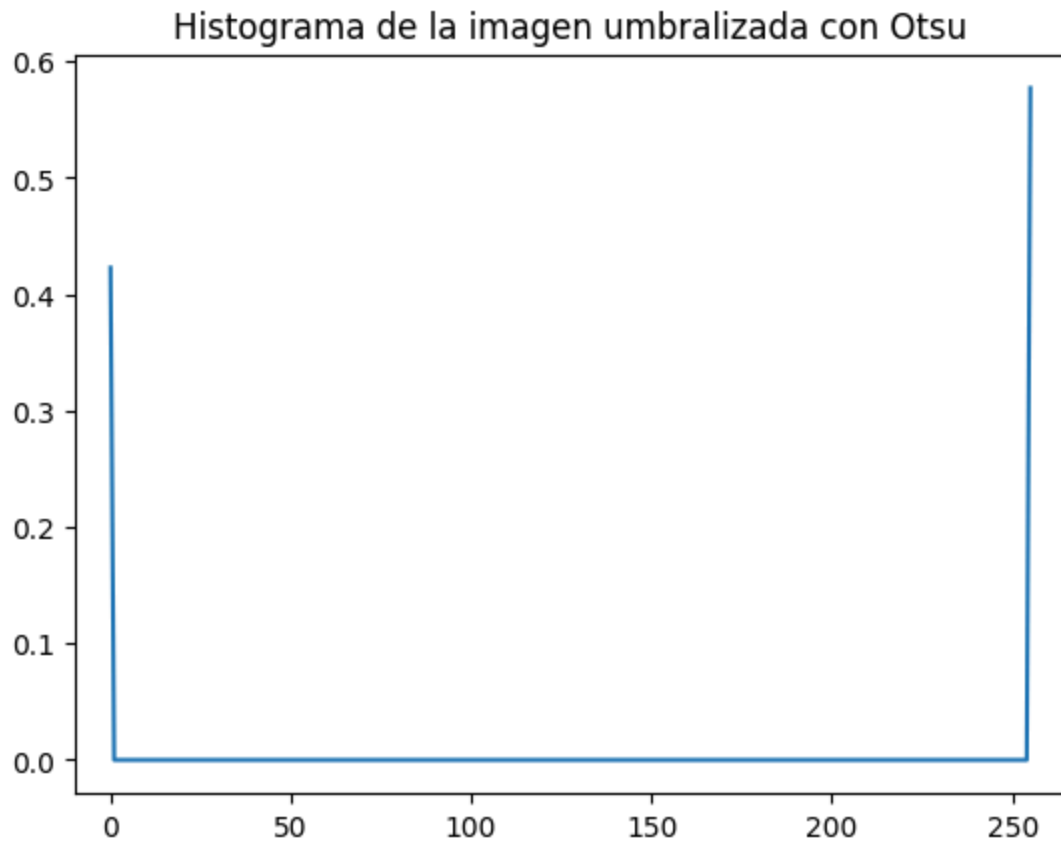
```
# visualizamos la imagen resultante
plt.imshow(otsu_image, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen umbralizada con método Otsu")
plt.show()

# visualizamos el histograma de la imagen umbralizada con Otsu
otsu_histogram = calculate_histogram(otsu_image, hist_title="Histograma de la imagen umbralizada con Otsu")
```

Valor de umbral automático (Otsu): 125.0

Imagen umbralizada con método Otsu





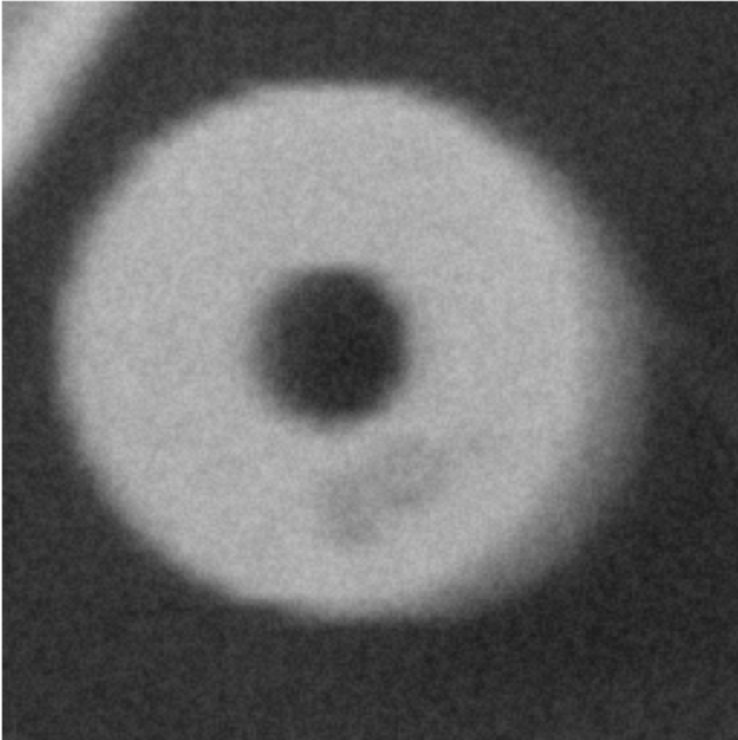
El método Otsu elige de manera automática el umbral a aplicar basándose en la varianza de los niveles de gris. La idea principal es separar dos clases de píxeles de tal manera que la varianza entre elementos de la misma clase sea mínima y, la varianza entre elementos de distintas clases sea máxima. En este caso, el método Otsu elige un umbral de 125, separando tanto las fibras oscuras del caso anterior, adicionando un poco de ruido debido a que se toman más valores de grises oscuros por debajo del valor umbral (detalles de fibras grises no visibles anteriormente). En conclusión, para este caso particular la umbralización simple ($S=85$) funciona de manera más eficiente para separar las fibras oscuras del resto de la imagen. No obstante, el umbral debe ser elegido de forma manual. Por ende, si se trata de una labor exhaustiva es recomendable utilizar la función automática (Método Otsu), mientras que, para tareas específicas se sugiere iterar manualmente el valor del umbral hasta obtener un resultado satisfactorio.

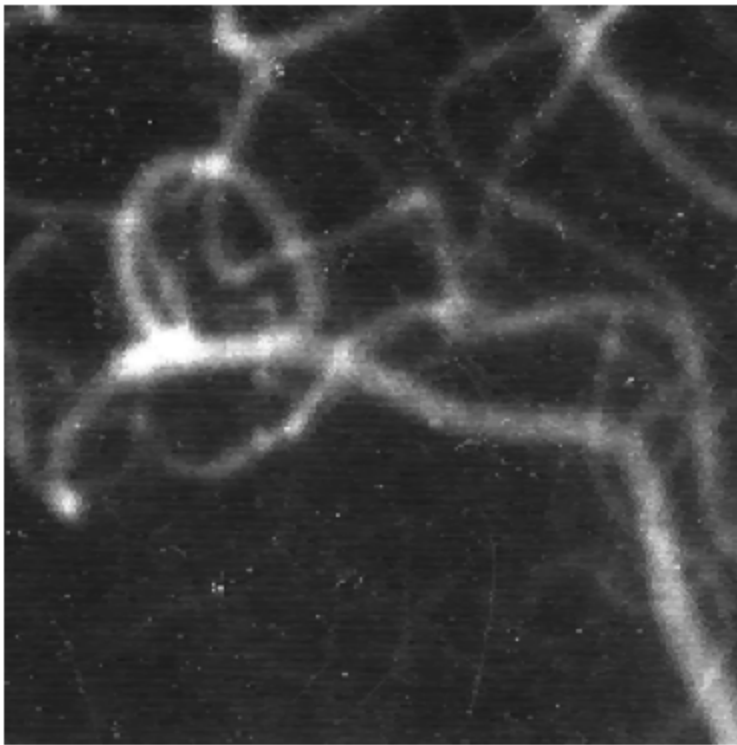
5. Umbralización doble

5.1. Cree una nueva sección en su notebook.

5.2. Cargue y visualice las imágenes de trabajo `**rondelle.png` y `**angio.png****`

```
In [ ]: # cargar y visualizar las imágenes
rondelle_image = read_and_show_image_gray_scale(path + 'rondelle.png')
angio_image = read_and_show_image_gray_scale(path + 'angio.png')
```





5.3. Se desea aplicar una umbralización doble de tal manera que la imagen resultante debe contener tres clases, cuyos niveles de gris son 0, 128 y 255 respectivamente. Para esto, debe utilizar las funciones de umbralización simple y umbralización doble.

```
In [ ]: def umbral_tres_clases(image:np.ndarray, low_threshold:int, high_threshold:int, clases:tuple=(0,128,255)) -> np.ndarray
        """
        Aplica una umbralización doble a una imagen para segmentarla en tres clases.

        Parámetros:
        image (numpy.ndarray): Imagen en escala de grises.
        low_threshold (int): Umbral inferior.
        high_threshold (int): Umbral superior.
        clases (tuple): Niveles de gris para las tres clases.

        Retorna:
        result_image (numpy.ndarray): Imagen segmentada en tres clases.
        """
        # umbralización doble para obtener la region entre los umbrales
```



```

img_middle = cv2.inRange(image, low_threshold, high_threshold)
# umbral sencillo para obtener la región por encima del umbral superior
_, img_upper = cv2.threshold(image, high_threshold, 128, cv2.THRESH_BINARY)
# la region fuera de los umbrales en img_middle tiene valor 0.
# la region con valores menores a high threshold tiene 0.
# para encontrar la región con valores menores al threshold inferior sumamos img_middle con img_upper
# de esta forma la región con valores menores al umbral inferior tendrá valor 0 + 0 = 0, la región entre los umbrales
img_final = img_middle + img_upper
return img_final

```

```

In [ ]: # aplicar umbralizaciones
rondelle_tres_clases = umbral_tres_clases(rondelle_image, 70, 140)
angio_tres_clases = umbral_tres_clases(angio_image, 70, 140)

# visualizar las imagenes
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
plt.imshow(rondelle_tres_clases, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen RONDELLE umbralizada en tres clases")
plt.subplot(1,2,2)
plt.imshow(angio_tres_clases, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen ANGIO umbralizada en tres clases")
plt.show()

# visualizar los histogramas
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
histogram_rondelle = calculate_histogram(rondelle_tres_clases, hist_title="Histograma RONDELLE umbralizada en tres clases")
plt.subplot(1,2,2)
histogram_angio = calculate_histogram(angio_tres_clases, hist_title="Histograma ANGIO umbralizada en tres clases", show=True)
plt.show()

```

Imagen RONDELLE umbralizada en tres clases

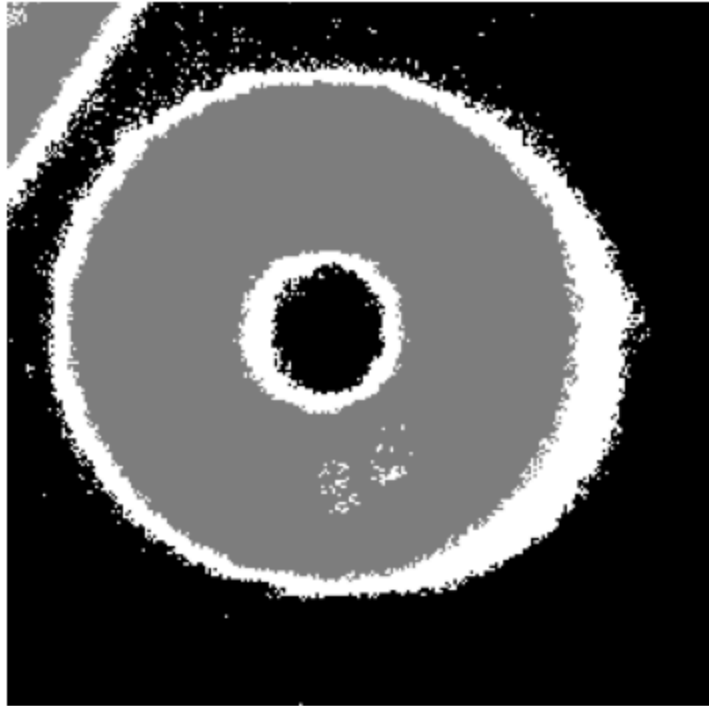
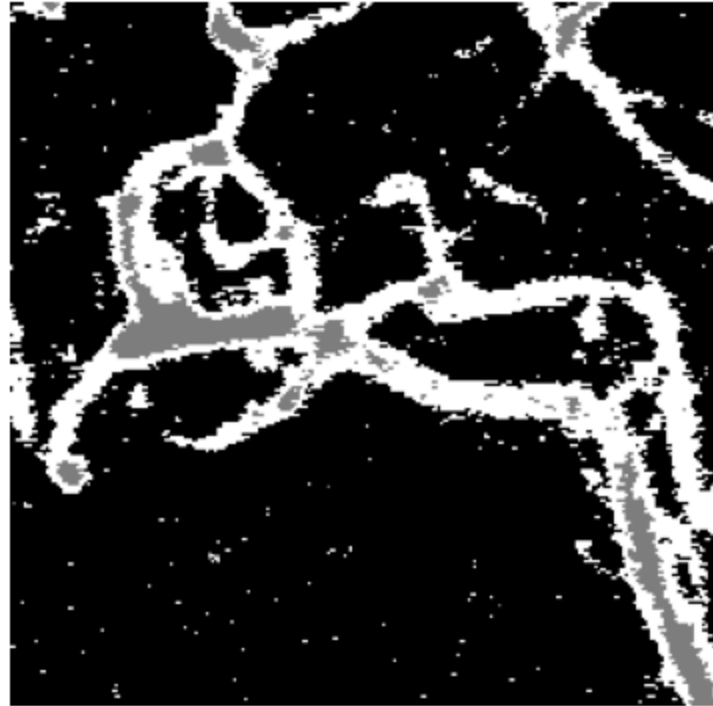
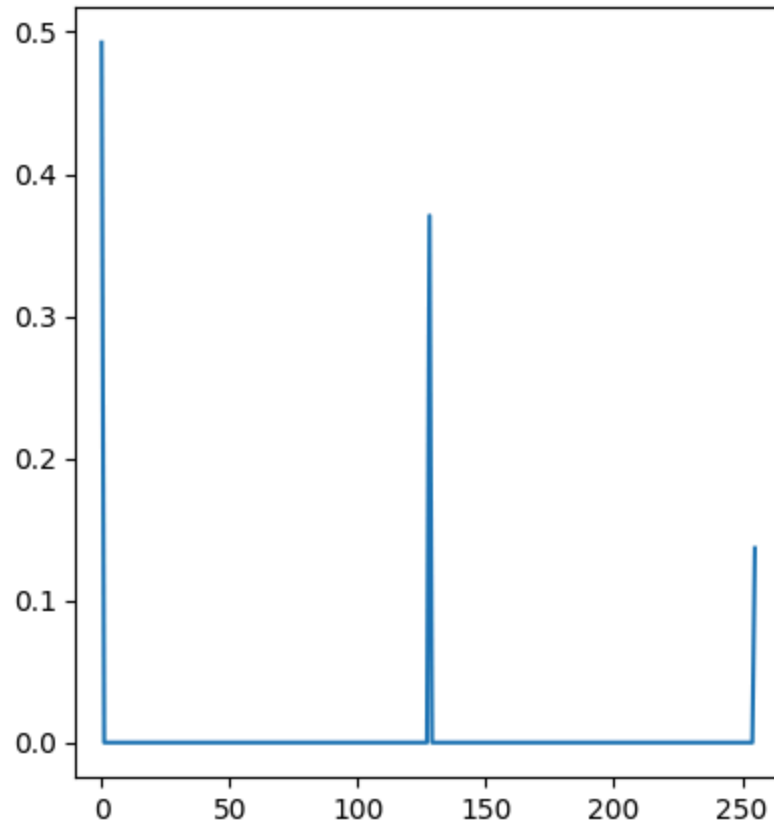


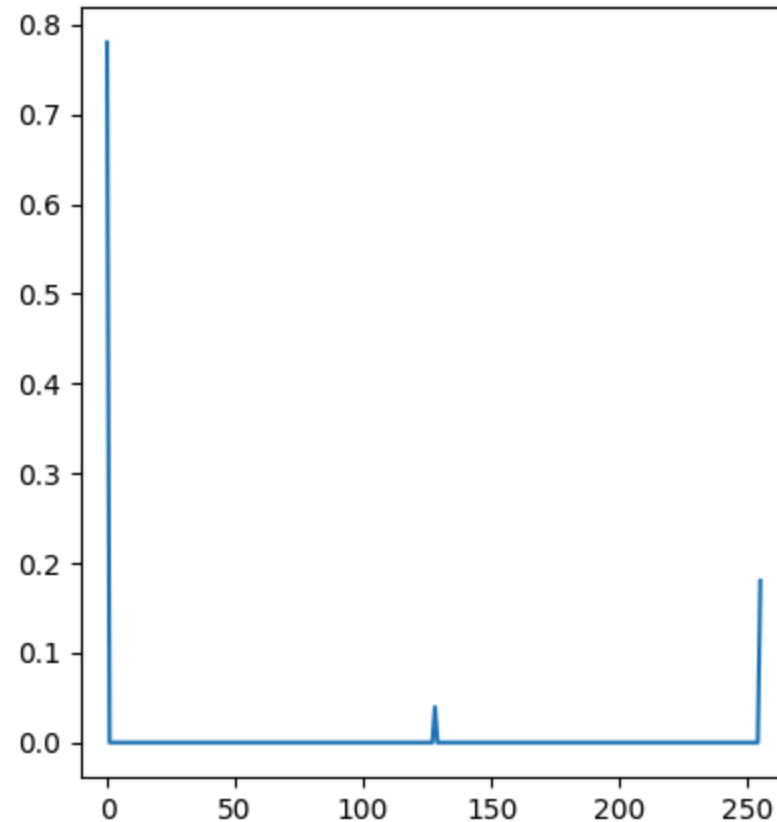
Imagen ANGIO umbralizada en tres clases



Histograma RONDELLE umbralizada en tres clases



Histograma ANGIO umbralizada en tres clases



5.5. Deduzca de estos ejemplos el rol y las condiciones de utilización de la umbralización doble. ¿Cómo es el comportamiento (ventajas y desventajas) con respecto a: conectividad, extracción de objetos de formas complejas o ramificadas y calidad de los contornos o fronteras de los objetos extraídos?

La umbralización doble permite clasificar píxeles dentro de un rango específico, mientras que, en la umbralización simple solo se puede clasificar por encima o por debajo de un umbral. Esto hace que la umbralización doble combinada con la umbralización simple sea una técnica útil para segmentar imágenes con objetos que tienen diferentes niveles de intensidad, permitiendo su extracción. Esta técnica puede ser muy útil en condiciones donde los objetos de interés presentan intensidades claramente diferenciadas entre ellos, pues es necesario escoger umbrales que faciliten dicha segmentación.

En cuanto a la conectividad de los objetos segmentados, esta técnica puede presentar limitaciones según los umbrales seleccionados, pues píxeles pertenecientes a la misma estructura pueden quedar clasificados en clases diferentes, presentando desconexiones en objetos

alargados o ramificados, como se puede observar en la imagen ANGIO.

Con respecto a la extracción de objetos con formas complejas, a pesar de que logra separar los objetos de las imágenes exploradas, puede resultar insuficiente si no se garantiza que los umbrales escogidos preservan la completa estructura del objeto que se quiere segmentar. En cuanto a la calidad de los contornos, esta técnica aporta una diferenciación abrupta entre clases, creando contornos poco suaves, pero muy bien diferenciados.

A pesar de las limitaciones mencionadas anteriormente, es una técnica sencilla de aplicar que puede ser muy efectiva para la segmentación de diversos elementos de la imagen.

6. Ejercicio de síntesis Taller 2.

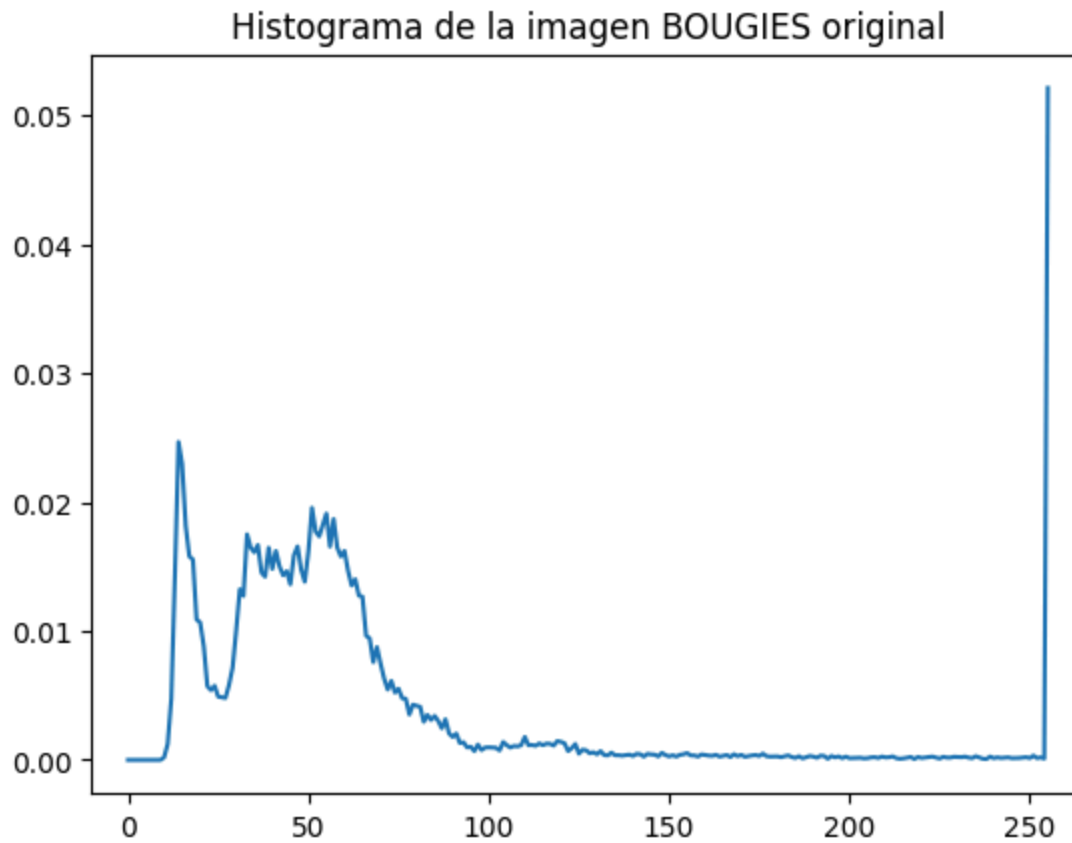
6.1. Ejercicio 1:

El objetivo de este ejercicio es poner en cero (NEGRO) las regiones que no pertenecen a las bujías (FONDO DE LA IMAGEN), mejorando al mismo tiempo el contraste de los objetos mismos (INTERIOR DE LAS BUJÍAS). Se puede utilizar CUALQUIERA de las operaciones vistas hasta el momento (operaciones aritméticas, lógicas, transformaciones del histograma, umbralización).

```
In [ ]: # cargar y visualizar la imagen y su histograma
bougies_image = read_and_show_image_gray_scale(path + 'BOUGIES.png')

bougies_histogram = calculate_histogram(bougies_image, hist_title="Histograma de la imagen BOUGIES original", show=True)
```





Se busca hacer dos procedimientos:

1. Segmentar las bujías del fondo y pasar el fondo a negro.
2. Mejorar el contraste de las bujías y los objetos que tienen dentro.

Observando el histograma y la imagen, se nota que el fondo de la imagen no solo toma colores blancos puros sino que tiene cierto ruido en tonos claros.

Para separar entonces el fondo, se puede **umbralizar** la imagen, tomando como umbral 50 (*este valor se calculó iterando*), de manera que todo píxel con valores por encima de 50 quedan en blanco y por debajo en negro, creando una especie de máscara binaria. Sin embargo, como se quiere que el fondo sea de color negro, se aplica la operación **NOT**.

A continuación se encuentra el resultado de la umbralización y la operación NOT.

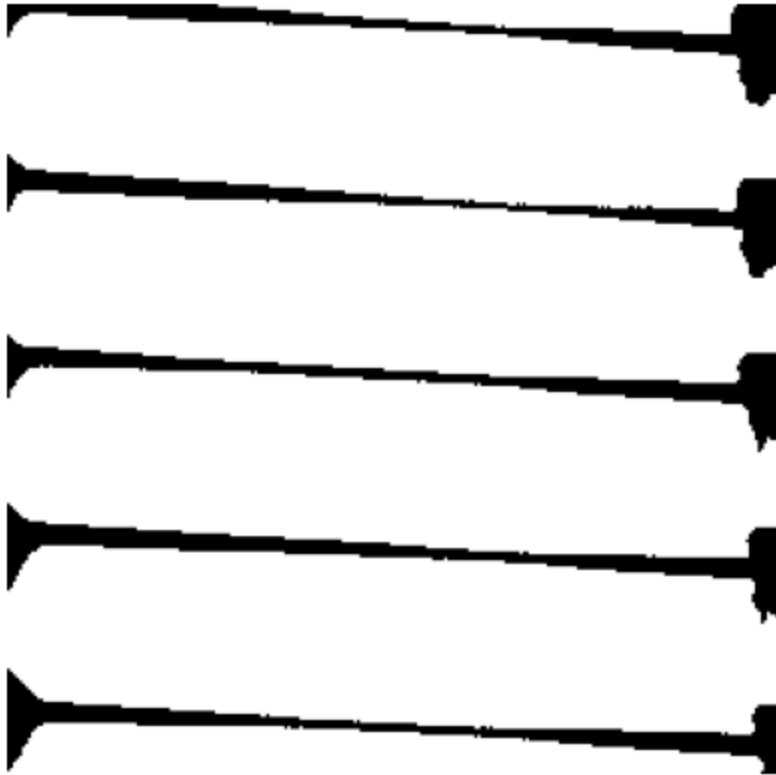
```
In [ ]: # umbralizar
_, thres_bougies = cv2.threshold(rescaled_img_bougies, 50, 255, cv2.THRESH_BINARY)
```

```

thres_bougies = cv2.bitwise_not(thres_bougies)

plt.figure(figsize=(10,5))
plt.imshow(thres_bougies, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.show()

```



El siguiente paso a realizar es juntar el fondo negro con las bujías haciendo uso de una operación **AND**.

```

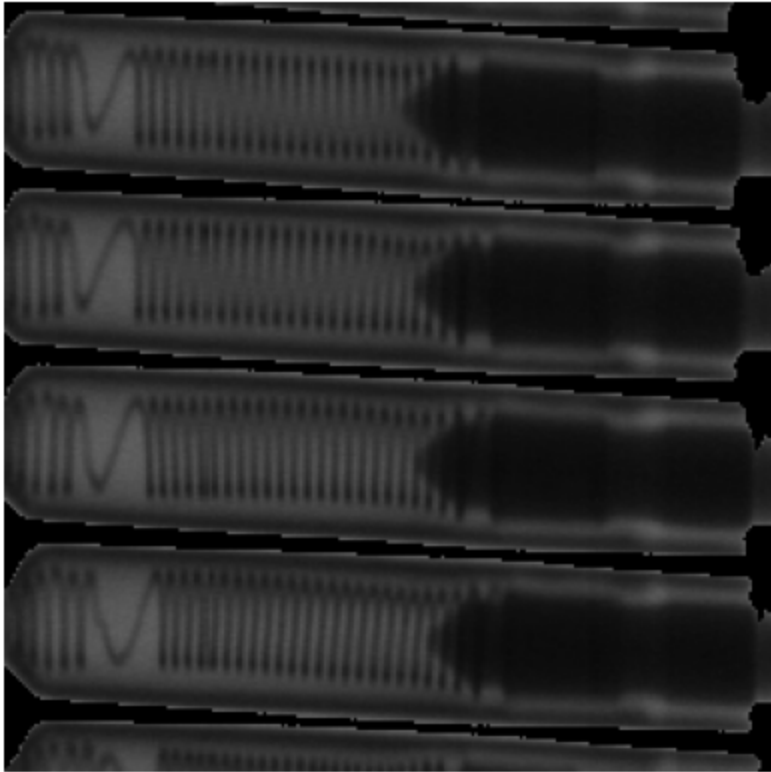
In [ ]: # bougies with black background
bougies_black_background = cv2.bitwise_and(bougies_image, thres_bougies)

plt.figure(figsize=(10,5))
plt.imshow(bougies_black_background, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Bujías con fondo negro")
plt.show()

histogram_black_background = calculate_histogram(bougies_black_background, hist_title="Histograma de bujías con fondo r

```

Bujías con fondo negro





Ahora que se tiene una imagen en donde la mayoría de píxeles toman tonos oscuros, es posible ecualizarla. Esto hace que el contraste mejore y, al ser una función 1 a 1, todos los tonos negros puros (0) serán mapeados nuevamente a este valor, por lo cual, el fondo de las bujías no variará pero si mejorará el contraste de los elementos en el interior de estas.

```
In [ ]: # ecualizar
equ_bougies = cv2.equalizeHist(bougies_black_background)

plt.figure(figsize=(10,5))
plt.imshow(equ_bougies, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.show()
```

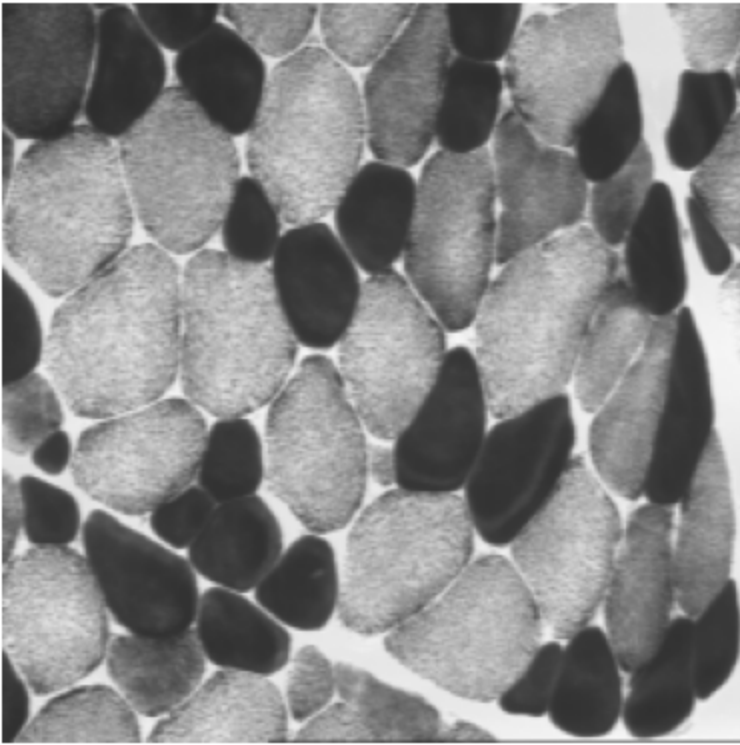


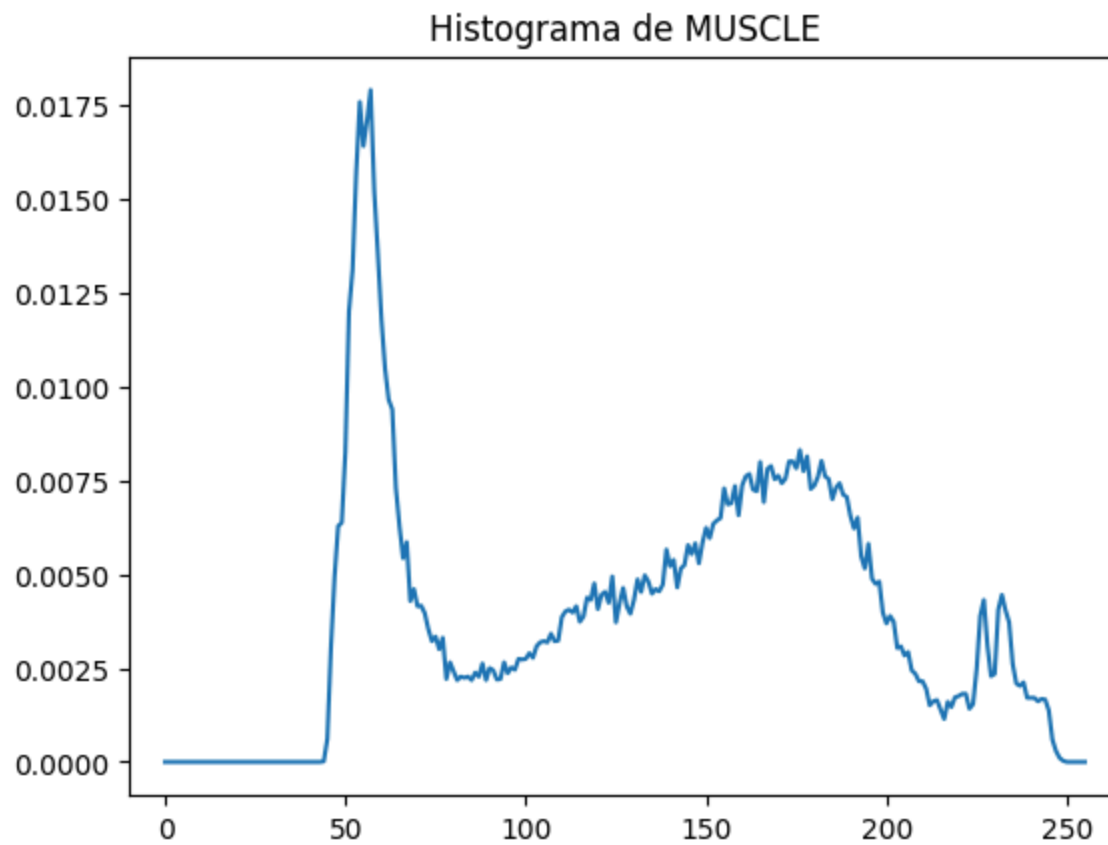
Finalmente, se obtuvo la imagen con un fondo negro y se mejoró el contraste de los objetos.

6.2. Ejercicio 2:

Repita el ejercicio de síntesis del Taller 1 pero esta vez usando umbralización.

```
In [ ]: # leemos la imagen MUSCLE y su histograma
muscle_image = read_and_show_image_gray_scale(path + 'MUSCLE.png')
muscle_histogram = calculate_histogram(muscle_image, hist_title="Histograma de MUSCLE")
```





La umbralización debe separar los valores más oscuros, los cuales, se encuentran entorno al valor 60. Se prueba umbralización con un umbral $S = 85$.

```
In [ ]: # umbralizamos
_, thres_muscle = cv2.threshold(muscle_image, 85, 255, cv2.THRESH_BINARY)

# visualizamos la imagen resultante
plt.imshow(thres_muscle, cmap='gray', vmin=0, vmax=255)
plt.axis('off')
plt.title("Imagen MUSCLE umbralizada con umbral 85")
plt.show()
```

Imagen MUSCLE umbralizada con umbral 85



Adicionalmente, se puede aplicar una función **bitwise not** a la imagen resultante para que las fibras oscuras tomen un valor de 255, y juntarlas con la imagen original haciendo **bitwise or**, de forma que las fibras blancas queden en blanco, y el resto de la imagen original se mantenga.

```
In [ ]: # aplicar not
        thres_muscle_not = cv2.bitwise_not(thres_muscle)

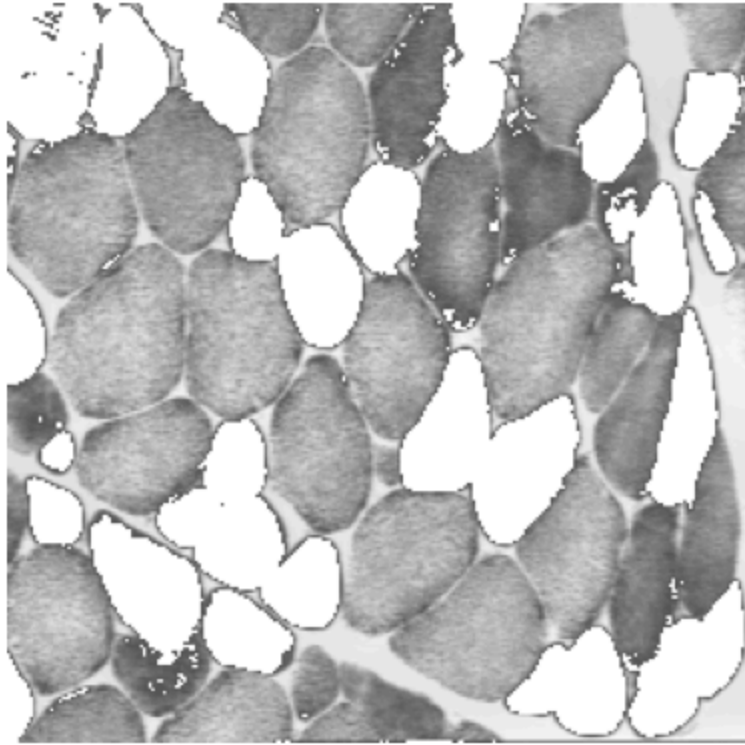
        # juntamos con la imagen original

        muscle_fibers = cv2.bitwise_or(muscle_image, thres_muscle_not)

        # visualizamos el resultado
        plt.imshow(muscle_fibers, cmap='gray', vmin=0, vmax=255)
        plt.axis('off')
        plt.title("Fibras musculares oscuras segmentadas con umbralización")
        plt.show()

        # visualizamos el histograma de la imagen resultante
        histogram_muscle_fibers = calculate_histogram(muscle_fibers, hist_title="Histograma de las fibras musculares segmentadas")
```

Fibras musculares oscuras segmentadas con umbralización



Histograma de las fibras musculares segmentadas con umbralización

